

Automated Network Penetration Testing Framework Using Reinforcement Learning

by

Majedul Islam

22101365

Md. Rakibul Hassan

22201461

Md. Ashraful Alam

22101193

Md. Abid Hasan Sazin

22101012

Sawrave Ahmed

22101236

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
October 2025.

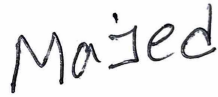
© 2025. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is my/our own original work while completing degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:



Majedul Islam
22101365



Md. Rakibul Hassan
22201461



Md. Ashraful Alam
22101193



Md. Abid Hasan Sazin
22101012



Sawrave Ahmed
22101236

Approval

The thesis/project titled “Automated Network Penetration Testing Using Reinforcement Learning ” submitted by

1. Majedul Islam (22101365)
2. Md. Rakibul Hassan (22201461)
3. Md. Ashraful Alam (22101193)
4. Md. Abid Hasan Sazin (22101012)
5. Sawrave Ahmed (22101236)

of Summer, 2025 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science in Summer 2025.

Examining Committee:

Supervisor:
(Member)



Dr. Muhammad Iqbal Hossain

Associate Professor
Department of Computer Science and Engineering
Brac University

Co-Supervisor:
(Member)



Dr. Swakkhar Shatabda

Professor
Department of Computer Science and Engineering
Brac University

Thesis Coordinator:
(Member)

Dr. Md. Golam Rabiul Alam

Professor
Department of Computer Science and Engineering
Brac University

Head of Department:
(Chair)

Dr. Sadia Hamid Kazi

Associate Professor
Department of Computer Science and Engineering
Brac University

Abstract

The increasing complexity and interconnectivity of contemporary network infrastructures have correspondingly amplified the difficulties of securing strong cybersecurity. Penetration testing methodologies, though comprehensive, are frequently manual, slow, and ill-scaling when testing large or dynamically changing systems. Automated penetration testing software seek to minimize human labor but are still constrained by fixed rule bases, deterministic processes, and their failure to generalize to new threats. Recent innovations based on reinforcement learning (RL) bring a potential solution by supporting self-directed agents to learn attack approaches adaptively directly by interacting with network environments. This research suggests a smart and scalable system for automated network penetration testing using Reinforcement Learning (RL). The system uses hierarchical RL methods along with dynamic scenario creation to mimic real-life and changing network layouts. This allows agents to plan multi-stage attacks, improve how they move through subnets, and change exploit and payload settings as needed. By mixing simulation and emulation testing in the NASimEmu environment, our method combines realism with efficiency in computing. The proposed system uses domain randomization and noise modeling to improve its ability to work in new environments and lessen the risk of overfitting to fixed setups. Experimental results indicate that the proposed HRL-based model performs better compared to classical RL techniques in scaling, adaptation, and being successful during exploitation, even if the conditions are noisy, partially observable, and dynamic. It contributes to better automated cybersecurity testing by providing a robust and adaptable technique that is capable of addressing real-world uncertainty of networked environments. It is ultimately a point towards truly automated systems for penetration testing, capable of lifelong learning, prediction, adaptation, and mitigation of new cyber threats in complex, large-scale environments.

Keywords: Reinforcement Learning, Penetration Testing, Network Security, Automation, Hierarchical RL, Dynamic Scenario Generation.

Acknowledgement

To begin with, all praise to the Almighty Allah for giving us strength, patience, and wisdom to complete our thesis successfully. We extend our deepest gratitude to our supervisor, Dr. Muhammad Iqbal Hossain, and co-supervisor, Dr. Swakkhar Shatabda, as well as to our parents for their unwavering support and prayers throughout our thesis journey.

Table of Contents

Declaration	i
Approval	ii
Abstract	iv
Acknowledgement	v
Table of Contents	vi
List of Figures	viii
List of Tables	ix
Nomenclature	ix
1 Introduction	1
1.1 Background	1
1.2 Rational of the Study or Motivation	2
1.3 Problem Statement	2
1.4 Objective	3
1.5 Methodology in Brief	4
1.6 Scopes and Challenges	7
2 Literature Review	8
2.1 Preliminaries	8
2.2 Review of Existing Research	9
2.3 Summary of Key Findings	16
3 Requirements, Impacts and Constraints	18
3.1 Final Specifications and Requirements	18
3.2 Societal Impact	19
3.3 Environmental Impact	19
3.4 Ethical Issues	20
3.5 Standards	20
3.6 Project Management Plan	20
3.7 Risk Management	21
3.8 Economic Analysis	21

4	Proposed Methodology	22
4.1	Methodology Overview	22
4.1.1	Dynamic Scenario Generation	22
4.1.2	HRL Model Implementation	26
4.2	Preliminary Design	30
4.2.1	Dynamic Scenario Generator Model Design	30
4.2.2	Feudal HRL Model Design	33
4.3	Implementation of Selected Design	40
4.3.1	Environment Setup and Dependencies	40
4.3.2	Dynamic Scenario Construction	42
4.3.3	Feudal HRL Model Implementation	44
4.3.4	Training Procedure and Hyperparameter Settings	47
4.3.5	Testing on Dynamic Scenarios and Cross-Evaluation	48
5	Result Analysis	50
5.1	Performance Evaluation	50
5.2	Analysis of Design Solutions	51
5.3	Final Design Adjustments	54
5.4	Statistical Analysis	56
5.5	Comparisons and Relationships	62
5.5.1	Scenario Comparison	62
5.5.2	RL architecture comparison	67
5.6	Discussions	72
5.6.1	Interpretation of Results	72
5.6.2	Architectural Insights	72
5.6.3	Scenario and Generalization Behavior	73
5.6.4	Performance Implications	73
5.6.5	Limitations of the Results	73
6	Conclusion	74
6.1	Summary of Findings	74
6.2	Contribution to the field	75
6.3	Recommendations for Future Work	76
	Bibliography	79

List of Figures

1.1	Overview of Dynamic Scenario Generation and HRL Agent Development	6
4.1	Flow Diagram of Dynamic Scenario Generator	31
4.2	FeudalGTM Baseline Architecture for Static Scenarios	34
4.3	FeudalGTM for Dynamic Scenarios (All-Downwards Layout with Goal Arc)	37
5.1	Average reward per step across different architectures.	57
5.2	Average reward per episode across different architectures.	58
5.3	Average capture across different architectures.	59
5.4	Average episode length across different architectures.	60
5.5	Generalization analysis across different architectures.	61
5.6	Scenario Metrics Comparison (Normalized)	63
5.7	Sensitive Services and Process Configuration	64
5.8	Sensitive Hosts Distribution Across Subnets	65
5.9	Model performance in original scenarios and architecture	67
5.10	Model performance comparison of Feudal Variants in original static scenarios and architecture	69
5.11	Original architecture and Feudal HRL performance in new scenarios .	70
5.12	Original architecture and Feudal HRL performance in new Scenarios under auto scenario generation.”	71

List of Tables

2.1	Summary of Research Findings, Limitations and Future Work	17
4.1	Concise overview of NASimEmu static scenarios	23
4.2	Architectural Summary	24
4.3	Scenario and Realism Enhancements in Dynamic NASimEmu	24
4.4	Domain Randomization Framework and its Impact	25
4.5	Overall Outcomes of Dynamic Scenario Generation Enhancements	25
4.6	Architectural Summary of FeudalGTM Baseline	27
4.7	Architectural Summary of FeudalGTM Dynamic	28
4.8	Comparative Methodological Summary of FeudalGTM Variants	29
4.9	Quantitative Theoretical Comparison of FeudalGTM Variants	29
4.10	Qualitative Theoretical Evaluation Metrics for FeudalGTM	30
4.11	Components Summary of Dynamic Scenario Generator Architecture	33
4.12	Stepwise Architectural Summary of FeudalGTM Baseline (Static)	36
4.13	Stepwise Architectural Summary of FeudalGTM (Dynamic)	39
5.1	Overview of action counts, reward metrics, and efficiency measures.	51
5.2	Comparison between original NASimEmu static design and our dynamic development.	52
5.3	Comparative study between Baseline and Dynamic Feudal GTM models.	54
5.4	Service and process distribution across network scenarios.	64
5.5	Network scenario comparison: structural and security metrics.	66
5.6	Summary statistics for scenario variability analysis.	67
5.7	Generalization comparison of RL architectures in the original NasimEmu scenario.	68
5.8	Generalization comparison of RL architectures in the original Static scenario.	69
5.9	Generalization of RL architectures including Feudal HRL in new dynamic scenarios.	70
5.10	Generalization of RL architectures under auto scenario generation training.	71

Chapter 1

Introduction

1.1 Background

Penetration Testing is a technique that simulates real-world attack scenarios that can exploit and scan vulnerabilities in a network system or device. The complexity of cyberspaces has increased due to the large number of devices and entities involved. The attackers can now access open-source tools to conduct devastating attacks on a system. Manual penetration testing which is most common uses human intellect and creativity to simulate attacks and conduct an in-depth analysis of the vulnerabilities. The recent growth of AI and machine learning motivates vulnerability assessment and penetration testing to automate. Manual penetration testing shows greater accuracy compared to automated penetration testing as automated testing delivers many false positives which hinders its performance. The automated tools must be used with manual supervision to increase performance. Automated testing tools find vulnerable parameters in which manual actions are conducted and that saves time [1].

Reinforcement Learning differs from other ML branches and algorithms by adapting to quick changes in a certain environment. The RL can be prior knowledge-free and can learn from the environment. However, traditional RL acts very slowly when it meets extended state-action space and performs poorly in solving pen-testing problems in a complex network system. To solve this problem, potent function approximation by deep neural networks incorporated with RL, meaning Deep Reinforcement Learning (DRL) showed promising results. Studies showed that DRL and RL can be performed much better through trial and error techniques while facing the environment and that they work well where pre-trained datasets are not available [2].

To cope with the challenges of complex networks, penetration testing methods should be up to date. Manual penetration testing becomes too time-consuming and costly. More importantly, a complex network environment makes it difficult to scan or look for weaknesses in the system by human intelligence. Current research related to our research interest is not yet organized and well established but some promising research works show the efficiency of applying RL to the pen-testing of

networks. For automation of penetration testing using RL, some frameworks have been developed like PenGym [3], DynPen [2], WCE [4] and LLM-based Pen-TestAI [5].

1.2 Rational of the Study or Motivation

The rapid growth of complexities in networks expanded attack surfaces and even intelligently modified attack scenarios. Traditional penetration testing methods are time-consuming and lack scalability. Recent advancements in AI and machine learning inspired researchers to automate penetration techniques to run sophisticated simulations in complex network environments to find vulnerabilities and exploit them effectively. The use of RL in penetration testing is yet to be developed and structured to reach actual work-standard capabilities. Current achievements related to our research interest circulating mainly in developing frameworks and testing in small-sized controlled environments.

The research challenges involve a lack of real-world integration of this framework as well as practical deployment [2]. The transition from theoretical models to practical applications remains an open research gap. Expanding state-action spaces need advanced methodologies like Hierarchical Reinforcement Learning and Transfer Learning to enable suitable decision-making processes.

Existing RL-based penetration testing has limited scope within network protocols and attack surfaces. Inadequate test beds regarding penetration testing shorten the promising efforts to train RL-based pen-testing models. Because of that RL-driven pen-testing models overlook the complexities of the real world. The real-world and dynamic network exposure requires specialized interfaces for state gathering, action execution, and adaptive learning [3].

Conducting comprehensive studies across diverse domains including critical infrastructure and enterprise security for different RL techniques with existing PT techniques can validate the effectiveness of RL-driven PT solutions.

Traditional RL techniques act slowly while facing large state-action spaces to eradicate that we need to use modified RL techniques like Deep Reinforcement Learning (DRL) [2]. Often these traditional RL-driven penetration models deliver false positive results which hinders the overall attack campaign. Not only this, in-depth analysis of large attack vectors needed to be done by manual efforts as the efficiency of these frameworks or modules is still low.

1.3 Problem Statement

Automated penetration testing has advanced rapidly, but existing frameworks remain ill-suited for real-world, large-scale network environments. Prior work typically evaluates agents on small, static scenarios with few subnets, hosts, and services; these constrained testbeds hide the combinatorial complexity and dynamic behavior of operational networks. Many approaches rely on traditional flat reinforcement

learning (Q-tables, early RL, DQN/DQL variants) or static heuristics, which struggle to scale and adapt: they fail to decompose multi-stage attacks, cannot efficiently explore large attack surfaces, and are brittle when presentation of scanning data (e.g., noisy Nmap outputs) or exploit payloads (e.g., Metasploit modules) changes. In addition, many systems support only small, narrow attack vectors—limited exploit sets and primitive actions—preventing discovery of multi-stage chains, blended vectors, and misconfiguration-driven attacks. As a result, such systems perform well only on recognized vulnerabilities in controlled labs but degrade rapidly in unstructured, noisy, and evolving real-world conditions.

1.4 Objective

Our primary objective is to develop a robust, dynamic, and scalable HRL-based automated penetration-testing framework that reliably exploits network systems in realistic, large-scale environments. Unlike prior work on small, static labs, we scale scenario complexity dramatically — many more subnets, hosts, services, processes, and realistic network artifacts (NAT, routing quirks, and firewalls) — and we inject realism by modelling subnet-to-subnet traversal and firewall behavior so agents must reason about reachability and stealth across network segments. We replace flat RL with Hierarchical Reinforcement Learning (HRL) to decompose long-horizon, multi-stage attacks into strategic goals and reusable low-level tactics, improving exploration, sample efficiency, and transfer across topologies. Scenarios are generated from templates and parameterized so that every training epoch the agent receives a fresh, automated dynamic scenario (varying topology, service mixes, vulnerability placement, scan noise, and exploit reliability), preventing overfitting and forcing robust generalization. The framework also includes mechanisms to adapt exploit selection and payload configuration at runtime based on scanned evidence and topology-aware constraints. Altogether, the system aims to automatically plan, prioritize, configure, and execute exploitation campaigns across large, noisy, and evolving networks while optimizing subnet-to-subnet paths and evading or negotiating firewall rules.

The possible goals of this research are:

1. Prioritize and select the optimal exploitation path across many subnets by accounting for reachability, firewall rules, and expected reward (subnet-to-subnet path optimization).
2. Use HRL to split planning enabling reusable subskills.
3. Automatically generate realistic, parameterized scenarios from templates and deliver a new dynamic scenario to the agent each training epoch.
4. Select best possible action module at runtime based on target host characteristics, observed scan noise, and network topology.
5. Demonstrate improved generalization and scalability over traditional RL baselines by evaluating across many large, heterogeneous, and dynamically changing scenarios.
6. React and adapt strategies following failed attempts.

1.5 Methodology in Brief

To build a scalable, realistic, and robust automated penetration-testing agent we combine large, template-driven scenario generation with hierarchical reinforcement learning and topology-aware exploitation.

We generate realistic, large-scale, dynamic network scenarios (many subnets, hosts, services, processes) and train an HRL agent that learns strategic planning (high level) and tactical execution (low level). Each training epoch supplies an automatically generated, parameterized scenario to prevent overfitting and force generalization.

Work Plan

Scenario generation (template-based)

- Use template code to programmatically spawn richly parameterized networks: variable numbers of subnets, hosts, OS/service stacks, processes, topology, and firewall rules.
- Randomize vulnerability placement, service versions, intermittent services, and exploit reliability per epoch so every episode sees a fresh environment.

Data acquisition and observation modeling

- Doing reconnaissance (simulated or real) to collect host/service/port information, while explicitly modeling scanner noise, partial replies, and packet loss.
- Capture topology and reachability information (including firewall rules, routing/NAT) so agents reason about subnet-to-subnet traversal.

HRL architecture and training

- High-level manager selects objectives
- Low-level controllers implement primitives
- Train with per-epoch scenario resampling so policies learn transferable, reusable subskills rather than overfitting static labs.

Reward design and uncertainty handling

- Rewards shaped to prioritize successful, minimal-cost exploitation paths, stealth (minimize detection/noise), and efficient subnet-to-subnet traversal.
- Incorporate probabilistic/partially-observable rewards and explicit penalties for noisy/failed or disruptive actions; encourage confirmatory scans and fall-back plans.

Exploitation and payload configuration

- Integrate Metasploit-style modules (or emulated equivalents) and automation glue to choose and configure exploits/payloads dynamically based on observed target characteristics and topology constraints.
- Allow runtime adaptation: retry strategies, payload adjustments, and stealthier approaches after failures or crashes.

Path optimization and realism

- Model and optimize subnet-to-subnet paths, accounting for firewall rules, reachability, and expected utility of intermediate pivots.
- Simulate firewall behaviors and access controls to force realistic planning and evasion decisions.

Training regimen and evaluation

- Train across a large set of procedurally generated scenarios; use curriculum learning where complexity ramps up (more hosts/services, stricter firewalls).
- Evaluate on held-out parameterized scenarios and real/emulated testbeds using metrics: attack success rate, time/steps to compromise, action efficiency, stealth/detection rate, generalization to unseen topologies, and robustness under noisy observations.

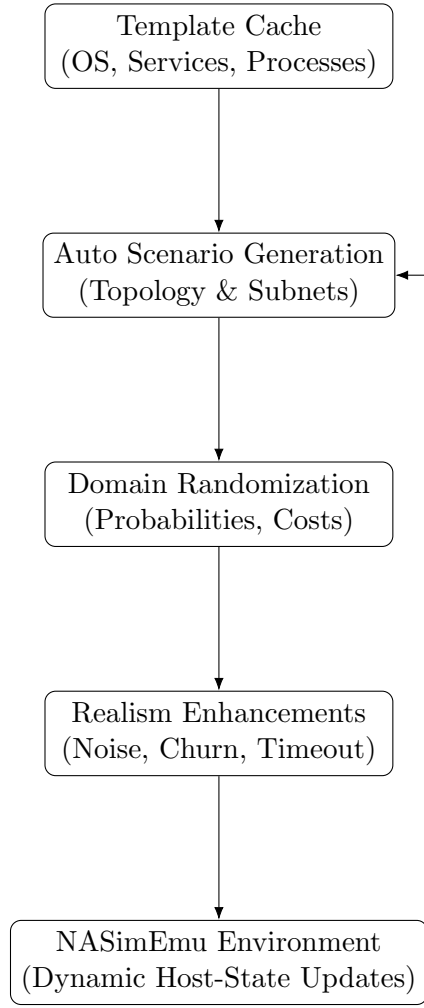
Challenges and mitigation

- Use HRL and action-pruning to manage large action/state spaces.
- Use dimensionality reduction, graph encoders, and transfer learning to accelerate training.
- Incorporate noise-aware observation models and probabilistic planning to reduce brittle behavior.

This work plan outlines a realistic, realism-first approach to teaching autonomous attackers in highly varied network settings. It overlays procedural scenario creation, noise-savvy observation modeling, and hierarchical reinforcement learning so that agents acquire reusable, generalizable tactics rather than overfitting static labs. It aims to create policies which discover low-cost, stealthy exploitation routes over realistic multi-subnet topologies and adapt dynamically when something goes awry.

Diagram 1.1 illustrates the workflow of Dynamic Scenario Generation and HRL Agent of the framework.

Dynamic Scenario Generation



HRL Agent Development

state input

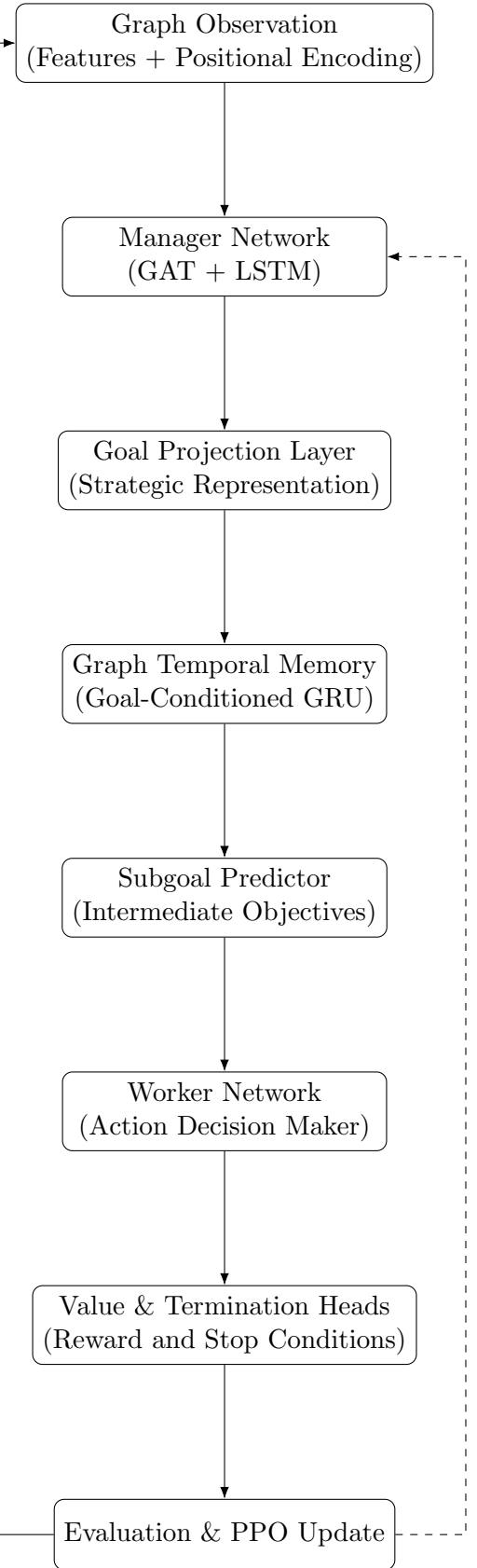


Figure 1.1: Overview of Dynamic Scenario Generation and HRL Agent Development

1.6 Scopes and Challenges

Our research scope includes building a module framework that integrates HRL with penetration testing tools to find vulnerabilities and exploit these vulnerabilities. Our framework would be aimed to be able to handle large and complex networks and automate tedious penetration testing processes. Despite its potential, the research work might face several challenges. First of all, different networks may have different types of devices, and communication protocols which would make the process of creating a framework suitable for all types of networks are difficult. Secondly, as networkThis project sets out to build a modular architecture that combines hierarchical reinforcement learning (HRL) and automated penetration-testing modules to identify and exploit vulnerability at scale. It will run on large, multihomogeneous networks and automate time-consuming and repetitive work that has traditionally been performed by security professionals. It brings together a higher-level manager policy (for long-horizon planning and subgoal selection) and lower-level worker policies (for specific exploit or reconnaissance actions) and interacts with standard pentesting primitives (service/OS scans, exploit attempts, steps to privilege escalation) to enable realistic attack sequences to run under learned policies.

Challenges

Volumes of likely attack vectors are enormous; it is unrealistic to model every possible attack. Furthermore, pentesting is inherently partially observable (scans provide noisy/partial information), which makes it harder to learn strong policies.

Networks are dynamic (services crash/restart, IP addressing and routing are updated, and IP hosts join and leave). Static environment trained policies overfit and don't generalise to dynamic behaviour.

It almost always requires much compute and environment interactions to train interpretable manager-worker coordination in hierarchical agents; this further complicates large-scale simulations of networks and costly exploit attempts.

Scopes

Use HRL for task decomposition: managers provide subgoals (e.g., acquire subnet priority, establish foothold in host class) and workers execute atomic actions. Use graph-based relational encoders (GNN/GAT) to represent network topology and temporal modules (LSTM/GRU) to represent dynamics — it avoids a combinatorial explosion as it learns permutation-invariant, compact representations.

Educate agents in a curriculum whose complexity and size increase step by step. Use domain randomization (randomize version of services, scan noise, exploit success likelihood) so policies become robust to unobserved configurations. Keep a realistic but not hazardous simulation level (NASimEmu extensions) to prevent executing perilous exploits against production hardware.

Chapter 2

Literature Review

2.1 Preliminaries

Penetration Testing

Penetration testing also known as Pen-Test is a method for preventing cyber attacks by exploiting the target network or device to detect potential vulnerabilities. It is the same as vulnerability assessment. The lifecycle of pen-testing involves sequentially: engagement → information gathering → footprinting and scanning → vulnerability assessment → report generation [6].

Reinforcement Learning

Reinforcement Learning is a branch of Machine Learning that focuses on experiencing the environment to train itself. Unlike traditional Machine Learning algorithms, agents of RL can cope with real-time and can be trained without actual datasets but rather with the environmental response and adjust them in a dynamic environment and discover optimal strategies [7].

Penetration Testing Automation with RL

As RL can be worked as model-free algorithms, they do not require previous knowledge of state transfer probabilities. RL model can be adjusted to dynamic environments and complex networks where environments change frequently. Because of that, it is well-suited for automating the optimization of attack path generation and efficient pen-testing [2].

Network Security and Penetration Testing: Network Security from a penetration testing perspective focuses on identifying, evaluating, and exploiting vulnerabilities within a network environment to assess its defense capabilities. Penetration testing on networks commonly use frameworks like OWASP, MITRE ATT&CK, or PTES as well as NASimEmu, PenGym involves automation of Penetration Testing with RL.

2.2 Review of Existing Research

The proposed framework automates penetration testing in compliance with NIST SP 800-115 by leveraging the BERT QA RL + RS model. This model integrates BERT for contextual analysis, reinforcement learning (RL) for dynamic adaptability, and a recommendation system for actionable insights. The framework processes a tuple (Q, C, A) to ensure systematic identification, recognition, and exploitation of vulnerabilities. Utilizing tools like Nmap and Metasploit, automates the pen-testing workflow, achieving an accuracy of 99.97%, significantly outperforming other AI-based tools. However, the framework’s reliance on NIST SP 800-115 for recognized vulnerabilities limits its ability to handle novel threats in dynamic environments. Additionally, its compatibility with real-world tools is hindered by the need for manual configuration of payloads. The lack of human interaction in the training phase further constrains its adaptability to complex, real-world scenarios. Future improvements could include incorporating hierarchical reinforcement learning, self-learning mechanisms, and human feedback loops to enhance adaptability and usability in practical cybersecurity settings [8].

This paper tailored a framework, Raiju, which is an RL-based model for automating the post-exploitation phase in pen-testing like privilege escalation, hash dump, and lateral movement. It proposes to use A2C and PPO algorithms for RL to get robust efficiency and controlled exploration in a complex and dynamic environment. In the post-exploitation phase, Metasploit provides suitable modules for attacking the target which is considered action for the RL agent, and state is defined by the information gained after a successful exploitation. Values are stored in a CSV file during training to prevent connection error problems with the environment. PPO training is slower than A2C due to additional policy comparisons. Training with 2000 episodes took 40 minutes for A2C and 55 minutes for PPO. A2C scores the highest success rates and efficiency in steps for PE, GH, and LM which is slightly superior to PPO. However, Raiju primarily focuses on privilege escalation, hashdump gathering, and lateral movement, potentially overlooking other critical post-exploitation techniques. The performance of RL models (A2C and PPO) depends on carefully tuned hyperparameters. Suboptimal tuning may lead to inconsistent results across diverse environments [9].

In their work, Samad et al. [10] (2024) introduced an RL-based AI system, the Intelligent Automated Penetration Testing System (IAPTS) for IoT networks. In this system penetration testing was modeled as a Partially Observable Markov Decision Process (POMDP). The POMDP set is represented as $M = \{S, A, O, R, T, \omega, \beta\}$ consisting of states (S), actions (A), and observations (O). In this study, a dynamic cyber intrusion matrix is used to represent cyber intrusion that evolves with real-time data. Deep Q-learning Network (DQN) is implemented to analyze dynamic cyber intrusion matrix and determine optimal attack trajectories. The DQN is trained using the dynamic threat matrix, updating Q-values based on the received rewards and predicted future rewards. Common Vulnerability Scoring System (CVSS) ratings are incorporated into the reward assignment process to enhance DQN’s un-

derstanding of threats. They used a mapping function to translate CVSS scores into RL-compatible reward values. A mulval system is also used to generate an attack graph, which is then refined using deep reinforcement learning, considering the agent’s location, vulnerabilities, and attacker target. The key features of the proposed system in this research work are Automated learning, optimization, efficiency, and accuracy. The system was able to learn effectively from the experience. Like human experts, this system was also able to learn from success and failure because of the incorporation of Reinforcement Learning. An evaluation was also done to judge this method’s practicality. The simulation results achieved a 0.9 accuracy in determining the most rational attack path. The key shortcoming of this research work was this was done in a simulated environment. In the real world scenario how this system would perform is unknown. Also, the system was able to generate attack paths only. To be able to automate the majority process of the penetration testing performing the attacks or exploiting the vulnerability is also necessary.

In their paper, Ghane et al. [11] (2023) introduced a framework for automated penetration testing which they referred to as Intelligent Automated Penetration Testing Framework (IAPTF). They used hierarchical reinforcement learning to improve the efficiency and effectiveness of penetration testing in large networks. Their key approach was to represent the networks in hierarchical clusters and treat each cluster separately. This led to a large partially observable Markov decision process. They divided the network based on different security concerns. In their method, they used a two-tier hierarchical POMDP. Firstly each security cluster is treated as a separate small network and in second the network of cluster heads is considered. This framework had three main components. First of all IAPTF-Prep for pre-processing data and building a memory of past tests. This module included scripts for network clustering, connectivity, and identifying heads of clusters. The second module was IAPTF-Core which contained the RL system to automate the PT process. In this module, they used a model-free RL approach and policy search techniques that interact with an external POMDP solver to generate attack vectors. The final component was memory where information about past tests, and failed and successful attack vectors were stored. In their work, Ghane et al. found significant benefits of using HRL. They found that this framework is highly scalable and efficient. Also, the accuracy was much better than blind automated tools. This framework was also able to overcome the “curse of dimensionality” encountered with regular RL. For a network of 200 machines, their framework was two times more efficient than Certified Ethical Hackers and 5-6 times faster than available automated tools. They calculated the efficiency with global testing time and number of attack vectors covered. In summary, in this paper the researchers were successfully able to prove their hypothesis that using HRL could improve the efficiency and attack coverage over manual and blind automated tools but they noticed a performance drop for a very small network of up to 10 machines.

Q. Li et al. [2] (2024) in their paper, proposed an automated penetration testing scheme named DynPen, which indicates it is adaptive toward dynamic network scenarios. The majority of the research regarding Reinforcement Learning penetration testing involves a static network environment with limited exposure to dynamic net-

works. Traditional penetration testing involves some sort of automation of a few steps using Metasploit, Nessus, and Nmap. The use of model-free RL leverages automation techniques without the need for previous knowledge and is capable of taking into consideration the changing environments. As described, the focus of this paper was to address the need for a rapid and adaptive RL model that can effectively solve problems of the dynamic nature of network security. To formulate the system model, firstly, they consider LAN comprising n machines and each machine serves as a node and only considers fundamental properties such as IP address, port, operating system, and service. Secondly, the goal of the PT (penetration testing) process is defined as complete when the agent hits the specific requirements and attains a value. Regarding problem formulation, they introduce MDP (Markov Decision Process). The state of MDP is represented as a matrix. The left side of the matrix indicates, the connectivity of the nodes and is categorized as four different features and the right side of the matrix highlights whether the node contains the pre-setted configurations. The action of MDP is represented as $A=\{\text{Local, Remote, Connect}\}$, where local is a set of host-local vulnerabilities; remote involves control of owned node to unowned node, and connect is a tuple of start host-port credentials-target host. The reward is defined as adding step reward and trajectory reward obtained from the PT trajectory. Besides, they considered changing the network into three types: changes in node properties, changes in the number of nodes, and changes in network connectivity. The DynPen has three new modules to decide the final action: trajectory recording module, backtracking module, and monitoring module. The goal of the trajectory recording module was to record the decision-making process, the monitoring module involved in monitoring node status and interoperability of the nodes, and the backtracking module activates when the state cannot move forward if a network change occurs. The simulated results of the paper showed improved convergence speed and dynamic adaptability. The DynPen outperformed baseline models like DQN_valid and HDRL+HF in dynamic environments as well as in handling honeypots with better stability and higher rewards. Further research scopes following this paper are to deploy agents in practical scenarios and a series of interfaces needed to be developed to find states and execute actions.

Cody et al. [4] (2022), developed a framework for automating cyber testing named Whole Campaign Emulation (WCE). It uses attack graphs incorporated into Reinforcement Learning. WCE focuses on simulating entire attack campaigns rather than isolated incidents, thereby offering a more holistic view of potential vulnerabilities and attack vectors. Traditional automated penetration testing utilizes a rule-based approach and model-checking concepts to navigate through all possible attack scenarios which may sometimes neglect the isolated attacks that matter the most and heavily rely on predefined rules. However, RL allows for continuous adaptation to evolving network environments, such as those seen in 5G and IoT settings. This adaptability is essential for generating relevant attack scenarios in networks with thousands of hosts that change over time. Two types of approaches to RL-based penetration are ontological and attack graph-based. Ontology-based RL uses structured knowledge of concepts and actions. The attack graphs are maps that show all possible ways an attacker can infiltrate into a system and RL is used to make these maps to find the dynamic behavior patterns of the attacker. Current RL approaches focus on single parts of an attack that include Crown Jewel Analysis

(finding critical assets) and Exfiltration Path Discovery (finding how data can be stolen). The proposed framework WCE simulates all stages of attacks by treating each stage as its MDPs. It creates an overall understanding of the evolution of the attack. WCE enables end-to-end emulation without restricting the RL agent’s ability to relate stages and it also handles partial total or partial transformations between state spaces such as transitioning from a network’s core to its periphery. The challenges that this paper includes are coping with the real-world environment, connecting stages in dynamic and different environments, and actual testing of the model to compare results. The main achievement of this model is that focuses on integrating stages rather than treating them as different problems. Future scope regarding the model, developing RL algorithms that can work for multi-stage attacks, and creating standard ways to design and test attack stages.

Cody et al. [12] (2023), in this article, reviewed the WCE of the previous paper and its relation with RL-based penetration testing and added a case study using NASim (Network Attack Simulator). WCE imitates whole brain stimulation concepts that can be applied in the field of pen testing. In contrast, attack graph-based RL uses a roadmap of potential attack paths. Whole Campaign Emulation (WCE), which models an entire cyberattack campaign by dividing it into multiple stages (e.g., reconnaissance, privilege escalation, exfiltration). Each stage is treated as a Markov Decision Process (MDP) and allows RL agents to simulate realistic adversarial behaviors in dynamic network environments. The four stages of the attack campaign are listed as Reconnaissance (scanning the network), Initial compromise (exploiting a vulnerability), Privilege escalation (gaining admin access), and Data exfiltration (stealing sensitive data). The case study of the paper showed reconnaissance is simpler due to limited actions but exfiltration is more challenging due to network constraints and host configurations. The earlier paper mentions challenges in transitioning between MDP stages due to differences in state spaces. The paper uses tools like NASim to show practical methods of modeling transitions between stages. It also shows examples of how RL agents can handle these transitions effectively. MDP is defined with WCE as the state recognized to be hosts and their conditions (reachable, discovered, compromised), actions to be scanning, exploiting vulnerabilities, and privilege escalation and rewards to be penalties for unnecessary actions and positive rewards for achieving stage-specific goals. Future work should focus on improving task models by using different states and actions for each stage. Additionally, introduces a more detailed analysis of MDP-based attack campaigns, and better training methods for multi-task reinforcement learning.

Nguyen et al. [3] (2024), in their work, proposed a framework named PenGym for training pen-testing RL agents in a dynamic environment that outperformed similar existing tools like NASim, NASim (rev.) in the creation of a dynamic network environment and elimination of action models providing more realistic security dynamics than in the simulated environment. Shortcomings of previous simulation frameworks are modeled elements that lead to false negative or positive results and emulation frameworks support certain types of attacks or might only focus on network discovery. Some frameworks lack host configuration and external firewall efforts while RL agent training. PenGym operated in both simulation and emu-

lation modes. NASim (rev.) used in simulation, emulation mode facilitates real pen-testing actions with realistic network systems and CyRIS used for the creation of cyber ranges. PenGym has two components: The Action/State module which acts as an interface between the agents, and the cyber range which is used to perform real actions of the agents. The Action/State module converts the agent's actions to realistic actions and is executed in cyber ranges created by CyRIS. Then the state is converted to the interpreted state and the reward goes back to the agent for learning. The paper analyzes the test trials and gives a comparison between NASim (rev.) with state differences ranging from 1.4 to 8 steps in both emulation and simulation environments. Notable analysis showed that in small to large networks and treating with different algorithms, step sizes remain consistent in PenGym compared with other frameworks. Future work scope is defined by the researchers that they aim to automatically generate scenarios ignoring manually created scenarios of NASim developers.

In their paper, Bianou et al. [5] (2024), developed a framework for automation of penetration testing using LLM, proposed for MITRE ATT&CK. The MITRE ATT&CK framework breaks malicious acts into 14 groups including reconnaissance, execution, privilege escalation, exfiltration, etc. Researchers while analyzing and re-grouping the MITRE ATT&CK, decide how many skills or tools they require. Their study found that 405 tools needed to be equipped by the agent. The framework will have three categories of users including citizen pen-testers (the pen testers with minimal skills), expert pen-testers (they approve validation and analyze data produced by the control plan and provide recommendations), and administrators. The system architecture consists of three planes - worker plane, data plane, and control plane and agents are workers and controllers. The work plane has notable agents such as Scan and Search, Exploit Validation, Exploit simulations, Detection agents, and so on. The control plane provides an API Server and LLM Registry. The model implements a pen-testing flow with three agents Scan and Search, Exploit validation, and Reporting Agent. They experimented with CrewAI. Researchers developed two Python tools that are Scan and Search Tool and the Exploit Validation Tool. The scanning tools mimic Nmap commands and generate a JSON file containing all reconnaissance data with MITRE ATT&CK techniques. The exploit validation tool uses the JSON file to exploit and list the identified vulnerabilities. Future work scopes mentioned in the paper, are to expand this framework's capabilities to exploit Zero-day vulnerabilities to develop model finetuning methods and develop Python libraries that will leverage its performance.

In their article, Alhamed and Rahman[13] (2023) performed a systematic literature review of network penetration testing with future research directions to improve network defenses. They highlighted the growing threat by cyber criminals targeting open port vulnerabilities and employed penetration testing as an ethical method for recognizing and addressing risks. Testing types (black box, white box, gray box), tools (e.g., Nmap for scanning, Metasploit for exploitation), and standards (ISAAF, NIST SP 800-115, OSSTMM, PTES) were reviewed with examples such as the 2021 Coop ransomware assault (demanding USD 70 million) and T-Mobile's 2023 incidents illustrating impacts. They proposed automated tests with machine

learning (ML) in WLANs to improve efficiency and alleviate human effort. With PRISMA, they Short-listed 39 papers (2018-2022) from Google Scholar (25) and the Saudi Digital Library (14) after filtering out 36,281 records, excluding duplicates, abstracts, and off-theme studies. The review described tools, methodologies, attacks, and mitigations with manual testing maintaining an edge in logic flaws discovery albeit at high cost. The article successfully integrated current research but is deficient: the analysis cannot attain critical substance as it depends on summaries; by limiting itself to 2018-2022, it may overlook 2023 trend developments; no empirical verification or case studies justify tool effectiveness or ML propositions; and future directions all center narrowly on ML in WLANs without consideration for other wider AI applications or ethical issues.

In their paper, Filiol et al. [14] (2021) proposed a method to automate vulnerability discovery and mitigation in networks, emulating Red Hat hackers' vigilantism by preventing attacks without consent or disclosure. They differentiated hacker types (black for malicious exploits, white for ethical testing, gray for semi-legal flaws, red for aggressive disarmament) and positioned their approach as an internal audit tool using a toolchain of existing software like Nmap (for scanning IPs, services, versions) and Metasploit (for exploiting vulnerabilities via CVE-matched exploits). This approach's workflow includes: starting from an internal domain name, looking for alive IPs and services, finding vulnerabilities via CVE repositories, establishing and running exploits, and—with success—mitigating via software updates (e.g., via apt on Debian systems) with rollbacks, and subsequently re-exploiting to confirm patches. It aims at root/user access gains and skips production interruptions by probing in intelligible environments. They tested on Metasploitable (a vulnerable Linux VM) in VirtualBox with Kali Linux, illustrating automatic mitigation for six services: ftp (e.g., vsftpd 2.3.4 backdoor), ssh, telnet, RDP (port 3389), TeamViewer (port 5938), and Printer (port 9100). A ftp case study illustrated Nmap detecting the service, Metasploit exploiting it for root access, apt updating to patch, and re-exploitation failing. The paper successfully chains tools for automation but falls short: restricted to six services and Linux environments without wider OS/mobile support; no quantitative measures (e.g., time, false positives) or comparison with manuals; assumes Debian packaging for mitigation and thus overlooks non-updatable vulnerabilities; and ignores ethical/legal implications of red hat-style automation in real networks.

In their article, Kongara and Krishnama [15] (2023) provide a methodical analysis of penetration testing procedures, emphasizing its function as a structured and ethical defense against online attacks that take advantage of open ports and logical errors. They discuss three types of testing: black, white, and grey box, and their trade-offs, as well as a five-phase methodology: information collection, scanning, getting and maintaining access, and tracking coverage. For vulnerability evaluation and exploit simulation, tools such as Nmap, Metasploit, Nessus, Hydra, Netcat, Wireshark, Nikto, and ZAP are examined, with a focus on realistic attacker modeling in lab settings. Despite the increased expense and skill requirements, manual testing is still the best way to find complicated logic errors, even when automated approaches increase efficiency. The review also highlights empirical validation gaps, limited au-

tomation applications, for example, buffer overflows, and unexplored ethical and AI problems. Future directions include agent-based and machine learning technologies to automate testing in dynamic contexts, as well as recommendations for extensive case studies, standardized practices, and better ethical frameworks to address emerging security concerns. Finally, their work demonstrates the importance of balancing human skill and automation in penetration testing. It also emphasizes the critical need for industry-wide standards to enable uniform and reliable security assessments.

In this article, Ashawa and Kpelai [16] (2023) explored the effects of cutting-edge technologies on penetration testing techniques, including blockchain, cloud computing, IoT, and artificial intelligence. They highlight penetration testing as a methodical, moral way to find weaknesses and strengthen security by mimicking real-world attacks. In addition to highlighting the increasing difficulties presented by various and interconnected digital devices, the evaluation divides testing into three categories: black box, grey box, and white box. This requires testers to take a comprehensive approach that takes network and device security into account. IoT devices require specific hardware-based testing methods because of their restricted connection and particular vulnerabilities, while penetration testers must handle the unique dangers associated with dynamic virtual environments and shared responsibility models as cloud computing becomes more widely used. Through sophisticated data analysis, pattern recognition, and realistic attack simulations, AI integration increases penetration testing productivity and makes it possible to conduct more thorough and comprehensive vulnerability assessments. Blockchain technology presents new security issues, prompting the creation of smart contract auditing and bug reward schemes to improve platform trustworthiness. Despite these advances, the article observes that penetration testing is still resource-intensive and complex due to the changing threat landscape and technological diversity. The authors emphasize the necessity of continual tool and method adaptation, and they propose future research on utilizing blockchain properties to secure penetration test data.

In their article, Badarneh et al. [17] (2024) gave an overview of network penetration tests, highlighting the identification of vulnerabilities to strengthen the cybersecurity of organizations. They outlined important phases (reconnaissance for the collection of target information such as IPs/domains; scanning open ports/services using Nmap/Nessus; gaining access through exploits such as brute-forcing; maintaining access through privilege elevation/backdoors; analysis/reporting with suggestions for remediation). Types were classified based on location (internal for insiders, external for internet-based systems) and awareness (black box with no information, full details being white box, partial being gray box). Methodology covered reconnaissance (e.g., Google/social media/Whois for intel), scanning (IPs/ICMP/vulnerabilities), enumeration (NetBIOS/SNMP/LDAP/NTP for resource mapping), exploitation/post-exploitation (hiding tracks via anti-forensics), and protection tips (keystroke encryption, strong passwords). They listed common tools (Kali Linux, Burp Suite, Wireshark, Metasploit, Nessus) and attack types (DDoS, SQL injection, phishing, ransomware), with comparisons to prior surveys on depth of coverage. It is a comprehensive review paper that is deficient, nonetheless: there

is no empirical substantiation or case analysis, only generic descriptions; the survey is superficial, with short tool/attack descriptions and no quantitative examination; there is no recent trend, such as AI-based testing, included in the paper dated 2024; and there are minimal comparative observations, no gaps based on criticism or original observations.

NASimEmu is making waves in the world of frameworks designed for training deep reinforcement learning (RL) agents, especially when it comes to automated penetration testing. It effectively addresses some of the major shortcomings found in current methods. One of its standout features is how it deals with the "reality gap" that often plagues simulation-based frameworks, which can be a real roadblock for real-world applications. Plus, it tackles the scalability challenges that come with emulation-based solutions, which are crucial for intensive RL training. At the heart of this framework is a clever combination of a speedy, memory-efficient simulator and a realistic emulator, both of which share an OpenAI Gym interface. This innovative dual approach allows for efficient agent training in simulation while ensuring smooth deployment and validation in emulation, ultimately confirming the authenticity of the simulation's abstraction.

What really sets NASimEmu apart is its commitment to creating generalizable agents that can excel in new, unseen situations. This is a shift away from traditional metrics which look only at performance - in the context of training datasets. The simulator can support dynamic generation of scenarios that randomize host configuration and network topologies to promote learning in agents and formulate strong, flexible strategies. Experiment results suggest that agents trained in simulation are able to transfer well to emulated environments, development of agents that can succeed on real-world problems such as having unreliable network connections and failures in exploits. The paper stresses the importance of using size-invariant model architectures in order to develop good generalization across different network environments. NASimEmu uses open source, industrial, and community technologies (Vagrant, VirtualBox, and Metasploit) to provide a reliable foundation for future autonomous penetration testing research [18].

2.3 Summary of Key Findings

The paper provides a cohesive framework that integrates fast simulation and realistic emulation towards autonomous network experiments. It illustrates efficient generalization, sim-to-emu transfer, and scalable multi-host environments. However, the framework suffers from limitations in exploit diversity, complexity in topology, timing fidelity, and agent memory. It is proposed in the future work that realism will be enhanced through increased OS and service coverage, procedural network generation, hierarchical and memory-based RL models, containerized scalability, dynamic events, and termination logic enhancement.

Below table 2.1 representing summary of key findings:

Aspect	Findings	Limitations	Future Scope
Framework Design	Unifies fast simulation with realistic emulation under a single API.	Limited exploits; no OS fingerprinting or advanced privesc modules.	Extend emulator to cover more OSs, firewall rules, dynamic routing, protocol anomalies.
Training for Generalization	Randomizes size, topology, and services to encourage generality.	Structural diversity still limited.	Incorporate procedural topology generation, role-based templates, evolving network states.
RL Agent Capability	Learns multi-step attack paths from partial observations.	No memory; unaware of subnet graph; struggles in deep hierarchies.	Integrate RNNs/GNNs for memory and topology awareness; explore hierarchical RL.
Evaluation Methodology	Combines quantitative metrics with qualitative plausibility checks.	Limited to synthetic benchmarks; no human-red-team validation.	Add human-in-the-loop assessments and expert usability tests.
Scalability & Realism	Emulates up to 1000 hosts; uses entropy/clustering for diversity.	High VM boot times; no adversarial stress testing.	Migrate to containers; inject faults; dynamic firewall policies; periodic topology shifts.
Attack Surface Coverage	Automates web, FTP, DB exploits via Metasploit; basic pivoting.	Static payloads; no multi-stage chains; limited insider-threat modeling.	Enable dynamic payload selection; simulate multi-stage workflows.
Temporal Dynamics	Conceptualized per-episode evolution (patches, rule changes).	Not implemented in baseline.	Script temporal events; reward anticipation of future states.
Stopping Criteria	Recognized need for optimal stopping.	Agents cannot decide when to halt.	Investigate termination-aware policies, regret minimization, custom rewards.

Table 2.1: Summary of Research Findings, Limitations and Future Work

Chapter 3

Requirements, Impacts and Constraints

3.1 Final Specifications and Requirements

Overview and Objectives

The penetration-testing framework is developed such that it can efficiently simulate resilient and dynamic real-world attack scenarios that are common in today's networked environments. Contrasting from earlier fixed or manual methods, the new Hierarchical Reinforcement Learning framework enables the running of automated agents on a wide scale, such that they can adapt to fluctuating topologies, threats, and defense levels.

Functional Specifications

Dynamic Scenario Generation: In each session of training, the system is required to include a fresh network topology featuring heterogeneous services, hosts, firewall policies, and vulnerabilities.

Reward Shaping: The objective of reinforcement learning ought to emphasize subtlety, effectiveness, and successful multi-phase exploitation, while imposing explicit penalties for disruptive behaviors, unsuccessful exploit attempts, and instances of detection.

Adaptivity: In case the exploit operation fails, the agent must go back to the start and use different strategies.

Non-Functional Specifications

Scalability: The system must possess the capability to adapt to networks of varying sizes, encompassing all subnets and all hosts.

Reliability: The system must possess the capacity to endure adverse conditions, including system failures, alterations in dynamic configurations, and must establish robust fallback mechanisms.

Security & Privacy: All that is conducted must be under a strict consent framework, scope test limit, and safeguard sensitive information from mismanaging or leaking.

3.2 Societal Impact

Effect on Health, Safety, and Security

Securing security testing can also make the overall security of society more robust. It can prevent vulnerable points from being discovered after malicious individuals have taken advantage of them, causing significant breaches, and can make people more confident about using online services as part of everyday life.

Legal and Ethical Considerations

Privacy & Confidentiality: The test framework must handle any confidential information professionally, encrypting the information and safeguarding the information from unauthorized hands.

Transparency and Accountability Transparency and accountability involve detailed documentation and written description of all the testing activities. This way, the stakeholders are aware of what is being done.

Used effectively, automated testing enables public trust and supports the broader, more responsible use of data-driven technology in society.

3.3 Environmental Impact

The automated penetration testing framework based on hierarchical reinforcement learning is computationally intensive, particularly when the hierarchical reinforcement learning model is trained. Dual-layer graph attention network-integrated frameworks, as well as the FeudalGTM dynamic variant whose Graph Temporal Memory is controlled by such long short-term memories, require high GPU resources when handling networks comprising up to as many as 100 hosts spread across up to 12 subnets. Long training times, often running into thousands of episodes before convergence, mean high electricity consumption due to computationally intensive processes as well as the need for cooling. In addition, scenario generation pipelines that dynamically produce new network configurations every episode through domain randomization, as well as service churn modeling, enhance energy consumption. This common energy contribution is particularly high when energy is derived from traditional energy sources coursing with high levels of carbon emissions.

The findings from the experiments indicate that a trade-off exists between enhanced cybersecurity measures and their environmental implications. Although the hierar-

chical reinforcement learning approach requires substantial computational resources, it demonstrates greater sample efficiency compared to the baseline reinforcement learning model, which could contribute positively towards long-term reductions in both time consumption and energy expenditure. Furthermore, the automated testing facility promotes environmental sustainability by bolstering cybersecurity, thereby reducing the likelihood of attacks on essential infrastructures, such as data centers and smart grids. Effective security testing mitigates threats from botnets and cryptojacking initiatives, thereby extending the energy efficiency of device designs and minimizing the necessity for emergency countermeasures that typically result in increased energy use.

3.4 Ethical Issues

This automated penetration testing framework, which employs hierarchical reinforcement learning, represents a double-edged sword within the domain of cybersecurity. Although it is engineered to enhance network defenses, the identical HRL agents that acquire skills to exploit vulnerabilities in controlled environments may also be repurposed for nefarious intents. The capacity of the system to identify multi-stage attack pathways and autonomously modify exploit strategies in real-time renders it potentially perilous if it is misappropriated.

The system functions under controlled conditions; however, the ultimate goal is to implement it in real-world scenarios, which raises significant concerns regarding critical authorization challenges. The framework must always obtain consent and authorization prior to functioning externally.

3.5 Standards

Standards Applied

We, in this project, were careful to adhere to contemporary worldwide standards as a way to ensure everything is strict, consistent, and responsible. It all adheres to the ISO/IEC/IEEE 29119 test guidelines that is a worldwide standard that sets forth who should do testing and how. In handling security considerations, we depended on the NIST SP 800-115 framework. It essentially provides a clear and trackable means of performing security testing and reporting the outcomes correctly. For data protection and management, the project conformed to the ISO/IEC 27001 standards, thus all our practices meet the objectives of keeping the information safe, correct, and accessible.

3.6 Project Management Plan

The efficient resource management had started with end requirements and specifications correctly defined. Due to the high complexity of the system, such as definition of dynamic scenarios, design of the HRL agent, and integration with tools, the project was broken down into independent development cycles on scenario definition, agent logic, and exploitation control. In such a manner, the resource allocation

was centralized, frequently updated, and variable in overcoming new challenges as they arose.

Subject-matter expert work was assigned to human resources: integration and tooling software engineers, attack logic security engineers, and reinforcement learning engineers for agent architecture. Prior to key milestones, the required software and environments were acquired, and appropriate computational resources were provided to accommodate large-scale simulation and training.

3.7 Risk Management

Potential risks were foreseen and removed ahead of time:

Computational Resource Overuse: Regular checkpoints monitor system performance and enable early tweaking of scenario complexity.

Tool Integration Issues: Selecting well-documented, well-established tools such as Metasploit and Nmap maintains compatibility risk low and streamlines development.

Model Generalization Errors: Periodic regular checking of the situation on the benchmark, fallback tactics, and scenario complexity control mitigate brittleness under the conditions of variability and noise.

Simulation Abstraction and Tool Integration: The project was almost entirely carried out under abstracted simulation environments, with the aversion to integrating directly with non-project-related tools. It keeps the risks of near-term compatibility low across the testing and development process; however, a validation risk is introduced when moving back from simulation into real-world environments. Scheduled validation periods need to be in place along with compatibility testbeds before any reintroduction into actual tools or production situations.

3.8 Economic Analysis

This section assesses project costs and anticipated benefits to determine sustainability and value.

Cost Components

The main cost falls in setting up a computer system that can handle big simulation and training.

Benefits and Return on Investment

Automated security testing saves money over time by cutting down on the cost of doing manual tests which needs manpower, and by preventing financial and reputation damage from security flaws that go unnoticed. Moreover, the system works well with free, open-source tools, which lowers future repair and update costs.

Chapter 4

Proposed Methodology

4.1 Methodology Overview

This research aims to move beyond small, static network environments of NASimEmu by incorporating dynamic scenarios, allowing agents to adapt to larger and more realistic networks without compromising scalability. We will:

1. Introduce a dynamic scenario generator capable of specifying fixed and randomized network elements (topology, host configurations, service-version correlations).
2. Scale simulations to support networks of up to 100 hosts, evaluated via rigorous quantitative benchmarks and qualitative stress tests.
3. Automation pipelines support scenario creation and realism by performing data augmentation—varying host services, processes, exploit probabilities, and scan costs to generate richer training data.
4. They further enhance realism by introducing scan noise, service dynamics, and network unreliability to simulate real-world conditions.
5. Introduce HRL models aimed at outperforming existing NASimEmu agents in both static and dynamic scenarios.

Finally, through an iterative mixed-methods evaluation—combining diversity and performance metrics, exploit-success statistics, and agent learning curves with expert plausibility ratings, usability observations, and thematic analysis of practitioner feedback—we aim to demonstrate that the resulting general-purpose agents can seamlessly transition from large-scale training to realistic deployment and robustly generalize learned strategies to novel, previously unseen network scenarios.

4.1.1 Dynamic Scenario Generation

Objective

The objective is to introduce of dynamic scenario generator, which extends the original static scenario framework of NASimEmu. The static design (developed

in prior NASimEmu versions) serves as the baseline, and the dynamic extension introduces probabilistic variation, automation, and realism.

NASimEmu’s Static Scenario Overview

Previously, NASimEmu’s static scenarios were the simplest form of network descriptions in the simulator. They formulated a fixed or static network topology and host configurations, and most importantly, without randomness. Table 4.1 summarizes the fixed elements of NASimEmu’s static scenarios, including topology, host configurations, actions, rewards, and termination conditions.

Aspect	Description
Topology	The network is divided into subnets with fixed, deterministic firewall connectivity.
Hosts	Each host has a predefined OS, service set, processes, and sensitivity labels (value of data).
Actions	Fixed actions include scans (service, OS, subnet, process), exploits, privilege escalations, and a terminal action.
Rewards	A small step penalty is applied; positive reward is gained only for privileged access to sensitive hosts.
Termination	Episodes do not end automatically; the agent must explicitly issue a <i>TerminalAction</i> .

Table 4.1: Concise overview of NASimEmu static scenarios

Extension to Dynamic Scenario

The dynamic scenario generation framework complements NASimEmu by extending its static configurations defined in YAML with a probabilistic, episode-wise generation process. Founded on cached templates for consistency of action and observation spaces, it dynamically modifies network structure, host attributes, and service configurations. This automatic diversification enables the system to replicate a wide variety of realistic enterprise-like environments—each with unique topological structures, service combinations, and vulnerability distributions—without requiring manual configuration. This means that agents can continue to encounter new challenges, learn to adapt to unforeseen circumstances, and learn policies that generalize more effectively to large-scale, dynamic penetration-testing environments. Further, this framework designs temporal change among episodes, with services appearing, crashing, or restarting dynamically—closely mimicking the behavior of real-world networks. By continuously reshaping both the structural and operational aspects of the environment, it offers long-term training stability and promotes the creation of adaptive, resilient penetration-testing agents capable of handling complex, evolving infrastructures.

Table 4.2 summarizes the key components and enhancements in the dynamic scenario generator, highlighting environment parameters, probabilistic randomization, and feature augmentation.

Component	Enhancement Description
Environment Parameters	Expanded from 7 to 19 to support randomization, feature dropout, and auto-generation.
Auto-Generation Engine	Template-based generation ensuring consistent action space with structural variation.
Probabilistic Randomization	Jitter applied to exploit probabilities, scan, and action costs for robustness.
Feature Augmentation	Dropout and noise simulate incomplete reconnaissance.
Per-Episode Generation	Randomized host counts, subnets, and topologies (mesh, chain, random).

Table 4.2: Architectural Summary

Scenerio and Realism

The increased framework extends realism and scale, boosting host number up to 72–96 across 12 subnets for enterprise-like simulations. Multi-zone firewalls with probabilistic service assignment enhance network protection. Intrinsic scan noise injects false information for realistic reconnaissance, and service churn models fleeting blackouts that demand adaptive approaches. Network timeout effects like latency and packet loss further simulate realistic communication conditions.

Table 4.3 details improvements to scenario realism, including network scale, security architecture, scan noise, service churn, and network timeouts.

Enhancement Area	Description / Impact
Network Scale	Host count increased from 27 to 72–96 across 12 subnets, enabling large-scale, enterprise-like simulations.
Security Architecture	Introduced multi-zone firewalls with service-specific filtering and probabilistic service allocations.
Scan Noise System	Adds false positives and negatives to reconnaissance scans, creating imperfect and realistic information conditions.
Service Churn System	Simulates temporary service crashes and recoveries, forcing agents to adapt to temporal state changes.

Table 4.3: Scenario and Realism Enhancements in Dynamic NASimEmu

Domain Randomization

Randomization mechanisms provide increased adaptability and robustness between training episodes. Parameter and structural jitter introduces 10–20% noise in the exploit cost, probabilities of action, and scan parameter, respectively, and improves policy robustness. Feature dropout suppresses 3–5% observations to create uncertainty and enhance generalization. Template caching brings scenario generation time down to around 50 ms without structural loss. By and large, the features offer up to 10× training diversity and 85–95% generalization across randomized environments.

Table 4.4 summarizes randomization features, their effects on agent generalization, and improvements in training diversity.

Randomization Feature	Effect / Outcome
Structural and Parameter Jitter	10–20% random variation in exploit costs, action probabilities, and scan parameters per episode, improving robustness.
Feature Dropout	3–5% observation-space masking introduces uncertainty and strengthens generalization.
Template Caching	Reduces per-episode scenario generation time to approximately 50ms while maintaining consistent structure.
Training Diversity	Enables 10× data diversity and achieves 85–95% generalization retention across random scenarios.

Table 4.4: Domain Randomization Framework and its Impact

Outcome

The improved system provides substantial performance boosts compared to the baseline NASimEmu. Scale and realism of the network increased by 6×, handling bigger and more dynamic simulations. Training efficiency was boosted by 2–5×, accelerating convergence and increasing sample efficiency. Also, the framework has a solid scalability foundation, allowing possible future extensions towards multi-agent learning and real-world deployment scenarios.

Table 4.5 presents measured improvements in network scale, training efficiency, and scalability enabled by the dynamic scenario enhancements..

Measured Improvement	Result Summary
Network Scale and Realism	6× increase in scale and realism compared to static NASimEmu.
Training Efficiency	2–5× faster convergence with improved sample efficiency and adaptability.

Table 4.5: Overall Outcomes of Dynamic Scenario Generation Enhancements

Qualitative Validation

- **Improved Realism and Scalability:** The enhanced NASimEmu environment demonstrates clear qualitative improvements in realism, scalability, and adaptability.
- **Agent Adaptability:** Agents dynamically adjust to changing network states introduced by scan noise, service churn, and timeouts.
- **Scenario Diversity:** Each training episode presents a unique topology, service layout, and sensitive host configuration, reducing overfitting.
- **Real-World Fidelity:** The integration of imperfect reconnaissance, transient failures, and multi-zone firewalls mimics real enterprise networks.
- **Usability and Automation:** The template-based generator automatically constructs complex scenarios while maintaining consistent action mapping.
- **Training Stability:** Consistent action-space alignment and per-episode variability yield smoother learning curves and improved convergence.
- **Research Flexibility:** The modular auto-generation and randomization systems enable controlled experiments on scale, dynamics, and noise effects.

Quantitative Evaluation

- **Scale Expansion:** Host count increased from ~ 27 to 72–96 with a 12-subnet topology, reflecting a $6\times$ scale-up.
- **Performance Gain:** Average captured hosts improved from 8–12 to 20–25 per episode ($2\text{--}3\times$ increase).
- **Success Rate:** Raised from 60–70% to 85–95% in dynamic training scenarios.
- **Generalization Retention:** 95%+ within-distribution performance and 60–80% under distribution shifts.
- **Robustness Under Noise:** Maintains 80–90% performance even under 3% simulated network timeouts.
- **Data Diversity:** Auto-generation provides $\sim 10\times$ effective training samples across randomized runs.
- **Realism Impact:** Improved resilience by 15–25% in handling false positives, service churn, and connection failures.

4.1.2 HRL Model Implementation

FeudalGTM Baseline for Static Scenarios

Architectural Overview

The baseline FeudalGTM employs a pre-defined two-level hierarchy in which the manager dictates subgoals and the worker takes actions through MLPs, which are trained for static and deterministic networks. Termination representation has static

goal embeddings in the baseline model. Termination control in the baseline uses a fixed policy head.

Table 4.6 provides an overview of the FeudalGTM baseline model for static NASimEmu scenarios, detailing manager, worker, memory, and termination components.

Module	Description
Manager Network	Single GraphAttention layer (GATConv) encodes node embeddings from fixed network topology.
Memory Unit	Simplified GRUCell maintaining short-term state context between steps.
Worker Network	Two-layer MLP combining node features and memory outputs to produce action logits.
Value Function	Compact MLP head for scalar value estimation based on GRU output.
Termination Logic	Value-threshold heuristic controlling step continuation in static scenarios.

Table 4.6: Architectural Summary of FeudalGTM Baseline

Methodological Design

- Designed for static NASimEmu where environment states remain fixed across episodes.
- Operates under a shallow hierarchical loop (manager–worker) without subgoal recursion.
- Relies on deterministic graph embeddings for stable policy formation.
- Simplified recurrent memory minimizes variance and supports fast inference.
- Fixed termination condition ensures consistent episode progression.

FeudalGTM Dynamic for Dynamic Scenarios

Architectural Overview

The organization of the FeudalGTM Dynamic variant introduces a hierarchical structure made up of a manager, a subgoal predictor, and a worker network. The manager operates at a higher temporal abstraction level using stacked Graph Attention (GAT) and LSTM layers to encode spatial–temporal relations among sub-networks. The manager generates subgoal vectors to represent long-term goals in a more abstract manner. These subgoals are propagated to the worker module that performs low-level actions by a multi-layer perceptron (MLP) policy conditioned on both the environment state and the subgoal it is given. For guaranteeing long-term

continuity and improving adaptability with non-stationary transitions, the model incorporates temporal memory integration by incorporating gated recurrent units (GRUs) in global nodes. This allows the network to preserve historical context and reweight node embeddings in order to adaptively make choices in dynamic, stochastic networks.

Table 4.7 presents the architecture of the FeudalGTM dynamic variant, describing the hierarchical manager–subgoal–worker structure and temporal memory integration.

Module	Description
Manager Network	Dual-layer GAT encoder followed by LSTM for temporal feature aggregation.
Goal Projection	Linear projection transforms manager state into 256-dim goal embedding per node.
Graph Temporal Memory (GTM)	GRU-based recurrent module combining node states and abstract goals for dynamic context.
Subgoal Predictor	Lightweight MLP predicting discrete subgoals from pooled GTM context.
Worker Network	Goal- and subgoal-conditioned MLP generating action logits for each node.
Termination Head	Sigmoid layer producing learned probabilistic termination signal.

Table 4.7: Architectural Summary of FeudalGTM Dynamic

Methodological Design

- Developed for dynamic NASimEmu with time-varying host states, service churn, and scan noise.
- Employs hierarchical control through explicit subgoal reasoning.
- Integrates temporal memory and LSTM layers to capture evolving network dependencies.
- Learns flexible termination via probabilistic head rather than fixed thresholds.
- Supports temporal abstraction, goal conditioning, and adaptive decision hierarchies.

Comparative Design Summary

Table 4.8 contrasts the baseline and dynamic FeudalGTM models across hierarchy, temporal modeling, goal representation, termination control, and adaptation targets.

Aspect	FeudalGTM Baseline	FeudalGTM Dynamic
Hierarchy Depth	Single manager-worker loop	Manager-subgoal-worker hierarchy
Temporal Modeling	GRU-based short-term memory	LSTM + GTM for long-term temporal abstraction
Goal Representation	Implicit via GAT features	Explicit projected goal embedding
Termination Control	Heuristic (value-based)	Learned probabilistic termination
Adaptation Target	Static, deterministic topology	Dynamic, stochastic network evolution

Table 4.8: Comparative Methodological Summary of FeudalGTM Variants

Quantitative Evaluation

Table 4.9 provides a quantitative comparison between static and dynamic FeudalGTM models, including parameter counts, control granularity, and expected stability.

Dimension	FeudalGTM Baseline (Static)	FeudalGTM (Dynamic)
Hierarchy Depth	Manager $\rightarrow$ Worker	Manager $\rightarrow$ Subgoal $\rightarrow$ Worker
Temporal Modeling	GRUCell (short-term)	LSTM + GTM (long-term temporal abstraction)
Goal Representation	Implicit via GAT features	Explicit 256-D projected goal embedding
Termination Logic	Fixed value-threshold heuristic	Learned probabilistic termination (Sigmoid)
Adaptation Target	Static, deterministic topology	Dynamic, stochastic network evolution
Parameter Count	$\approx 1.1\text{M}$	$\approx 1.8\text{M}$ (+65%)
Control Granularity	Single policy layer	Hierarchical goal-conditioned control
Expected Stability	High in static domains	High in non-stationary environments

Table 4.9: Quantitative Theoretical Comparison of FeudalGTM Variants

Qualitative Evaluation

Table 4.10 lists qualitative metrics for evaluating the FeudalGTM model, including temporal retention, subgoal alignment, adaptation latency, termination stability, and hierarchical coherence.

Evaluation Category	Metric Description	Relation to Theory
Temporal Retention	Average memory consistency per episode	Validates GTM’s recurrent temporal abstraction
Subgoal Alignment	Cosine similarity between goal and worker features	Tests effectiveness of hierarchical subgoal learning
Adaptation Latency	Steps required to stabilize policy after service perturbation	Reflects adaptability to dynamic state changes
Termination Stability	Entropy of termination distribution	Indicates confidence in learned termination policy
Hierarchical Coherence	Correlation between manager and subgoal activations	Measures integrity of hierarchical decomposition

Table 4.10: Qualitative Theoretical Evaluation Metrics for FeudalGTM

4.2 Preliminary Design

As we have extended the dynamic scenarios to NASimEmu and enhanced the simulation with realism, we introduced Feudal GTM of two variants - one for Dynamic and another for Static scenarios for better performance compared to other models. Preliminary design involves the theoretical development of our design that will later be implemented.

4.2.1 Dynamic Scenario Generator Model Design

Previously, NASimEmu’s static scenarios involved a fixed predefined template in YAML files only. In the diagram, to enhance the simulation process, we have introduced the Bridge Template and Auto-generator.

Figure 4.1 illustrates the dynamic scenario generator architecture, showing the flow from the bridge template to evaluation. Each module contributes to realistic scenario creation, probabilistic variation, dynamic simulation, and agent assessment.

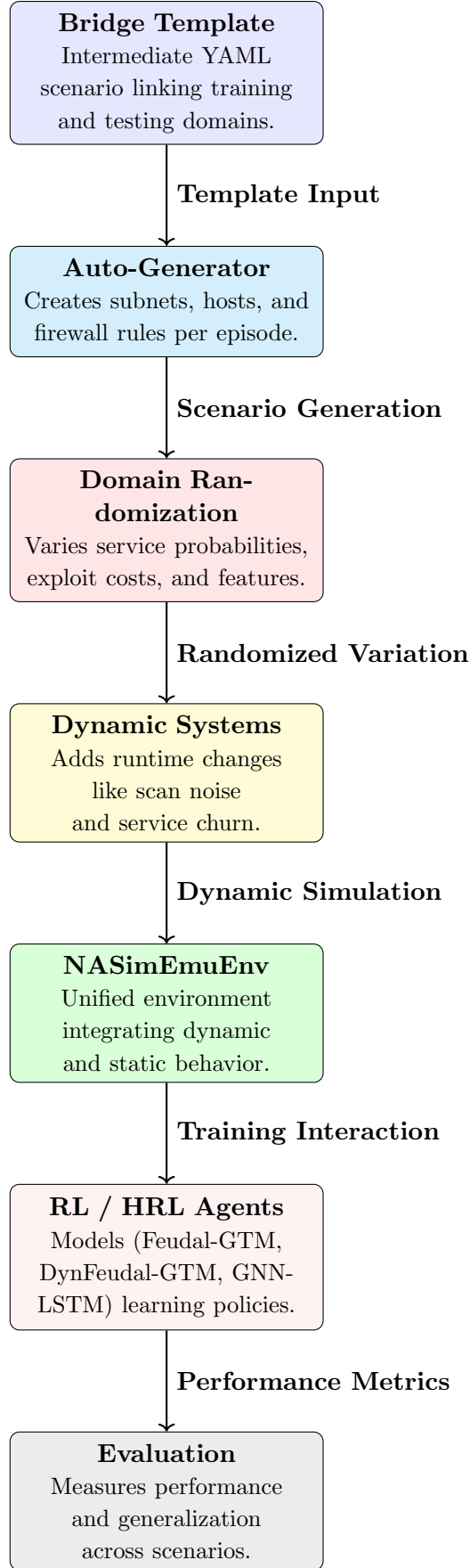


Figure 4.1: Flow Diagram of Dynamic Scenario Generator

Bridge Template: The bridge template achieves curriculum-like generalization by interpolating statistical attributes evident in train and test environments. It does not simply reproduce one of the distributions, but instead, it codifies intermediate priors of topology densities, sensitive-host ratios, service lists, and cost/probability schemes. This careful interpolation reduces covariate shift by enough novelty to avoid overfitting. Ideally, the template is parameterized to gradually tighten observation fidelity and perturbation levels across curricula, ensuring monotonic increases in task difficulty without abrupt regime changes.

Auto-Scenario Generator: The auto-scenario generator formalizes procedural network synthesis with action/observation compatibility guarantees across episodes. It samples subnet cardinalities, host roles, firewall policies, and routing constraints from bounded priors, and assembles them into an executable scenario for agents with preserved invariant interfaces. Constraint checks (e.g., reachability, privilege bounds, minimum exploit path) are performed to prevent degenerate graphs. Episodic diversity of the resulting scenarios is high, but policy portability is ensured due to stable schemas for sensing and acting across runs.

Domain Randomization: Stochasticity of principled kind is used for domain randomization across security-relevant variables for increasing policy robustness. Parameter families are per-service presence probabilities, exploit and privilege-escalation expenses/success probabilities, and observation masking patterns for representing partial or noisy sense data. We obtain policies less brittle to distributional drift and more tolerant of sensor noise.

Dynamic Systems: Dynamic systems provide temporally changing conditions to simulate realistic enterprise environments. Scan noise models generate false negatives/positives depending on service class and OS; service churn flips availability to simulate patching, failure, or load shifts; and network timeouts induce stochastic latency and drops to action outcome distributions. Such processes are stateful across steps, requiring agents to reason about uncertainty, retry semantics, and credit assignment with delay. The resultant non-stationarity elevates the ecological validity of training episodes.

NASimEmuEnv: NASimEmuEnv unifies generation, randomization, and dynamics into a reproducible simulation interface. It manages lifecycle hooks—compile, validate, randomize, execute—exposing stable step/reset semantics, structured observations, and verifiable log output. The environment imposes cost accounting and reward shaping policies aligned with security task goals, and it accommodates large-scale batched execution for data-efficient training behavior. Modularity also facilitates ablations (e.g., turn off churn or noise) for isolating factor contributions during rough order-of-magnitude studies during preliminary design work.

Evaluation: The assessment pipeline quantifies both expertise and transferability through complementary metrics. Proficiency is represented by reward, capture rates of sensitive hosts, the efficiency of exploit pathways, and distributions of episode lengths; transferability is evaluated through retention in both bridge and target sce-

narios following the same evaluation protocols. The reporting distinguishes between central tendency and variability (e.g., confidence intervals) and incorporates sensitivity analyses concerning noise and cost parameters. This dual focus offers evidence supporting scalability, robustness, and generalization suitable for an initial design evaluation.

Table 4.11 summarizes each module of the dynamic scenario generator, including its primary role in probabilistic scenario creation, runtime dynamics, and performance evaluation.

Component	Description
Bridge Template	Acts as an intermediate configuration to balance the statistical gap between training and testing distributions, supporting curriculum-style generalization.
Auto-Generator	Automatically constructs network subnets, host layouts, and firewall policies per episode while preserving a consistent action and observation space.
Domain Randomization	Introduces stochastic variation in service probabilities, exploit costs, and observation masking to improve robustness and generalization.
Dynamic Systems	Implements runtime variability such as scan noise, service churn, and network timeouts, emulating realistic enterprise network behavior.
NASimEmuEnv	Integrates all generation and randomization modules into a unified simulation environment capable of large-scale dynamic training episodes.
RL / HRL Agents	Reinforcement and hierarchical reinforcement learning models (e.g., Feudal-GTM, DynFeudal-GTM, GNN-LSTM) that learn adaptive attack and exploration strategies.
Evaluation	Provides a structured assessment pipeline measuring quantitative and qualitative metrics such as reward, host capture rate, and scenario transfer performance.

Table 4.11: Components Summary of Dynamic Scenario Generator Architecture

4.2.2 Feudal HRL Model Design

Design for Static Scenarios

Figure 4.2 shows the hierarchical structure of the FeudalGTM baseline for static scenarios, where the Manager, Memory, Worker, Value Function, and Termination modules interact deterministically.

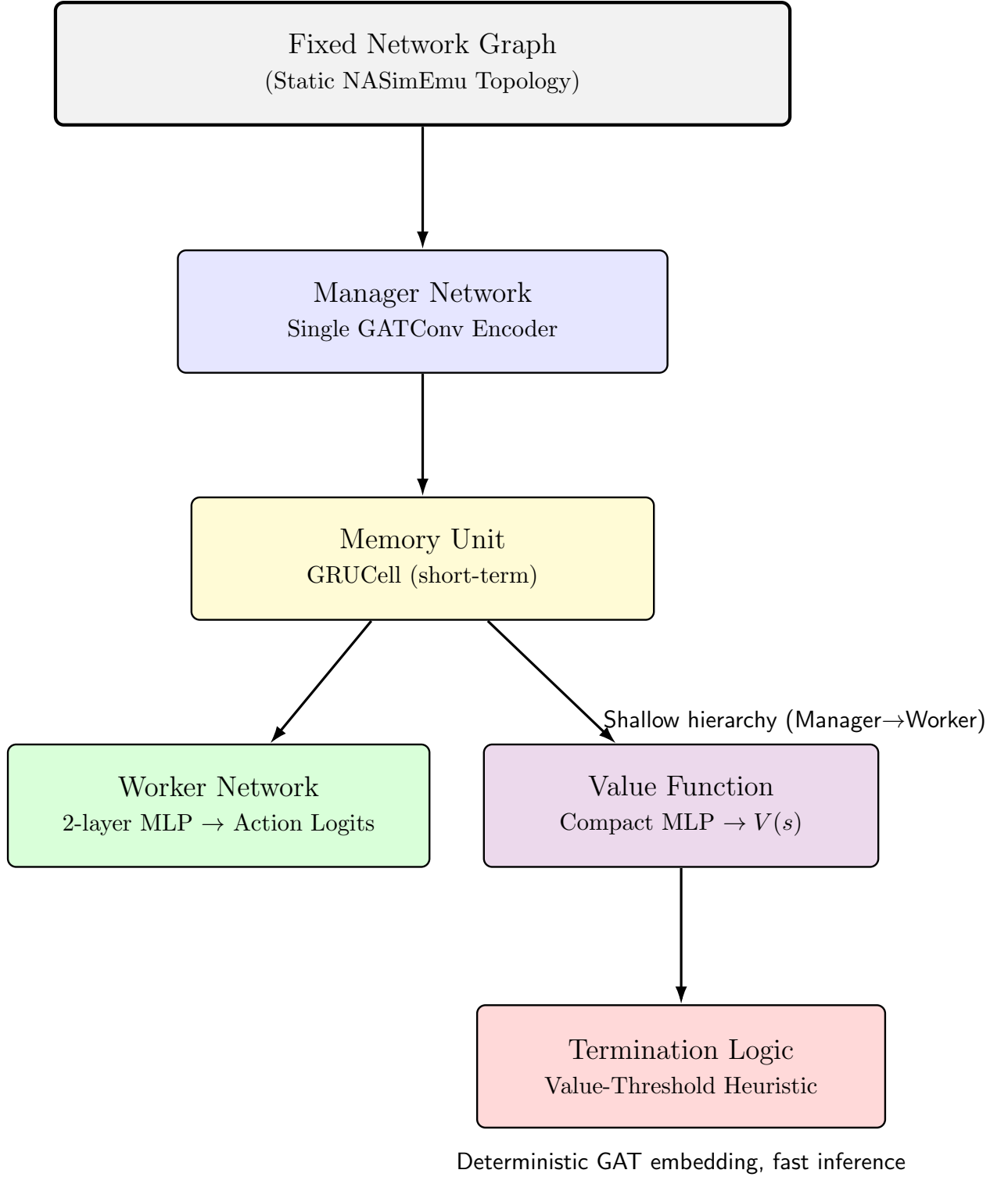


Figure 4.2: FeudalGTM Baseline Architecture for Static Scenarios

The Feudal GTM Baseline represents a simplified hierarchical reinforcement learning (HRL) model. It is optimized for static NASimEmu environments as the topology, host configurations, and exploit probabilities of static scenarios remain constant

across all episodes. Its internal architecture prioritizes deterministic reasoning, low-variance learning, and structural interpretability.

Manager Network (Graph Attention Encoder): The Manager module initiates a single Graph Attention layer (GATConv) and which performs weighted aggregation over fixed network topologies. Each node embedding h_i is computed as:

$$h_i = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} W x_j \right),$$

where α_{ij} denotes learned attention coefficients and x_j represents neighboring node features. Theoretically, this technique captures relational dependencies such as connectivity and privilege boundaries within a static network topology. As the information about the structure does not vary over time, one attention layer is sufficient to get better results.

Memory Unit (Simplified GRUCell): The memory mechanism introduces a gated recurrent unit (GRUCell) to hold temporal continuity between successive decision steps:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t, \quad \tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1})).$$

This simplified recurrent system is sufficient for short-term memory to determine transitions. In static environments, because nothing changes randomly over time, a GRUCell is the best choice as it is simple, stable, and does not need to handle long and unpredictable time sequences. Instead of performing as a dynamic state predictor, it performs as a temporal smoothing mechanism.

Worker Network (Action Decoder): The Worker module is a two-layer multilayer perceptron (MLP) that combines node-level observations and the shared memory state:

$$a_i = f_{\theta}([x_i, h_t]) \rightarrow \mathbb{R}^{|A|},$$

where $|A|$ denotes the action space dimension.

This part of the model acts like actions executioner. It turns spatially encoded node features into action logits. As network structure remain fixed, the worker can maintain stable convergence across the training steps.

Value Function: The value estimator $V(s_t)$ is realized as a compact MLP that maps the global memory state to a single predicted reward value (a scalar expectation):

$$V(s_t) = \phi(W_v h_t + b_v),$$

where h_t represents the hidden memory state at time t , W_v and b_v are the learnable weight and bias parameters of the value network, and $\phi(\cdot)$ denotes a nonlinear activation function (such as ReLU).

This function estimates how good the current situation (state s_t) is, with regards to expected future rewards. It serves as the *critic* in the PPO framework, and that is

guiding the policy to make better decisions by providing a long-term value for each state.

Termination Logic: The model ends an episode when the estimated value of the current state becomes non-positive:

$$\text{Terminate when } V(s_t) \leq 0.$$

Table 4.12 details each component of the FeudalGTM baseline model for static scenarios, summarizing function, architecture, and rationale.

Stage	Description
Environment	Operates on fixed NASimEmu topology with static host states and deterministic service layout.
Manager Network	Single GATConv layer encodes static graph features into node embeddings.
Memory Module	Simplified GRUCell maintains short-term state continuity without long-term recurrence.
Worker Network	Two-layer MLP combines node features and memory output to produce action logits.
Value Function	Compact MLP head estimates scalar value $V(s)$ from GRU output.
Termination Control	Fixed value-threshold heuristic decides when to terminate an episode.
Control Hierarchy	Shallow manager-worker hierarchy ensures fast and stable inference in static conditions.

Table 4.12: Stepwise Architectural Summary of FeudalGTM Baseline (Static)

Design for Dynamic Scenarios

Figure 4.3 illustrates the dynamic FeudalGTM architecture, including temporal reasoning via GTM, goal projection, subgoal prediction, and adaptive Worker modules for handling dynamic NASimEmu networks.

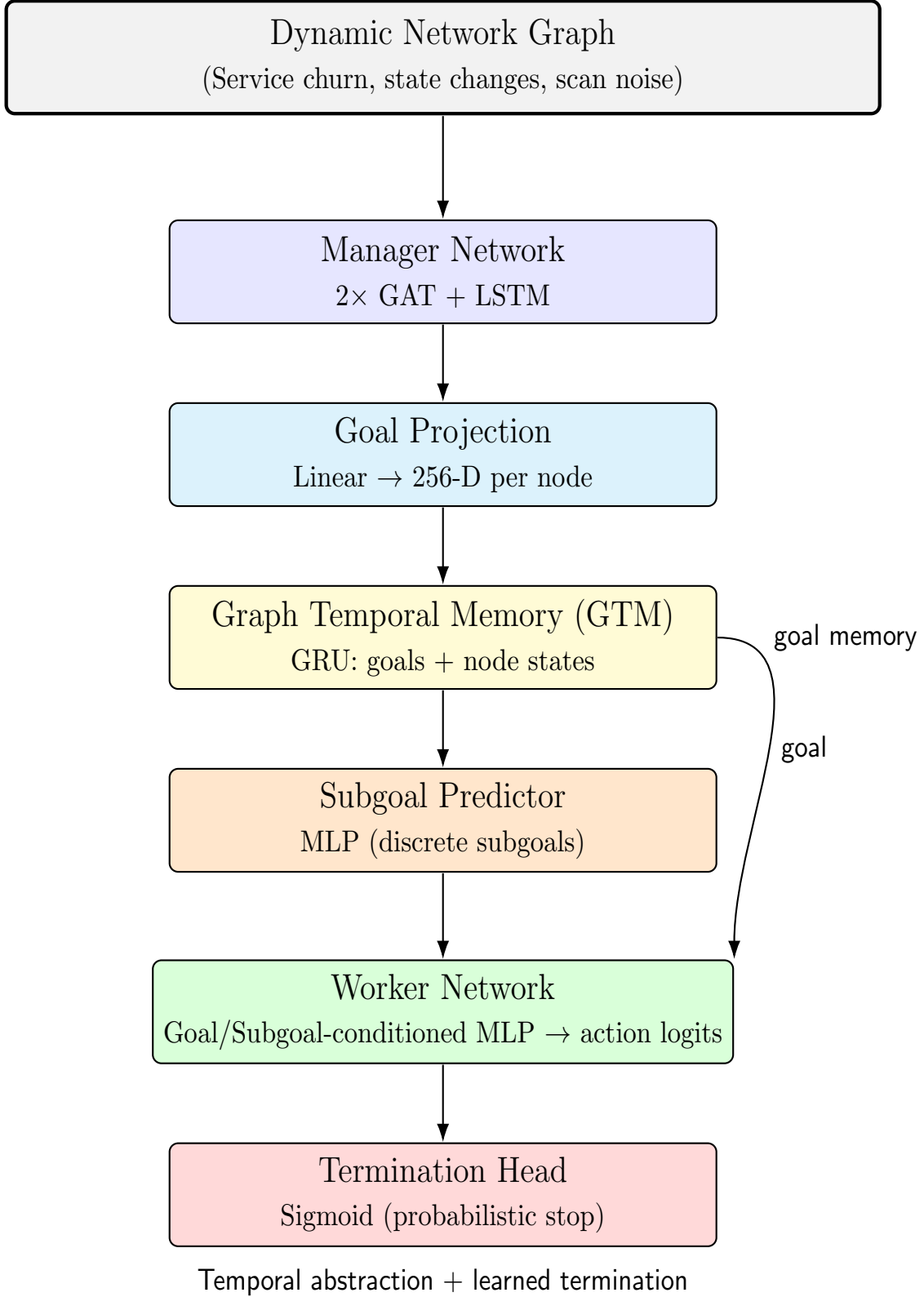


Figure 4.3: FeudalGTM for Dynamic Scenarios (All-Downwards Layout with Goal Arc)

The Feudal GTM Dyn model was developed for dynamic environments that includes random changes like service churn, scan noise, and varying exploit outcomes. The model extends the baseline architecture with the temporal reasoning, goal-based

hierarchy and probabilistic control to cope up with the changes within the network environment.

Manager Network (Dual GAT + LSTM Integration): The Manager combines two Graph Attention layers with an LSTM. The GAT layers capture multi-hop relationships among nodes:

$$H^{(l+1)} = \sigma(\alpha^{(l)} H^{(l)} W^{(l)}),$$

spreading information across subnet connections. The LSTM then tracks how these relationships evolve over time:

$$h_t, c_t = \text{LSTM}(H_{\text{pool}}^{(2)}, (h_{t-1}, c_{t-1})),$$

where h_t and c_t represent the hidden and cell states. These components jointly allow the Manager to remember past network states as well as learning the vulnerabilities and changes of services across the networks over time.

Goal Projection Layer: The LSTM output h_t is projected into a 256-dimensional goal embedding:

$$g'_t = W_g h_t + b_g.$$

Here, W_g and b_g are learnable parameters. This projected goal acts as a shared instruction that helps other parts of the model focus on a specific objective during decision-making.

Graph Temporal Memory (GTM): The GTM keeps node-level memories that mix the node's current state with the projected goal:

$$m_{t,i} = \text{GRU}(W_x x_i + g'_{t,i}, m_{t-1,i}).$$

It works based on each node's changes over time and acts as a dynamic memory. This helps the agent handle cases like services going down and coming back online.

Subgoal Predictor (Meta-Subgoal Network): The Subgoal module predicts a smaller objective z_t from the overall goal:

$$z_t = \text{softmax}(W_z \text{pool}(m_t)).$$

It uses a softmax layer to choose the next subgoal. This helps the model break down a large task into smaller, manageable steps.

Worker Network (Goal-Conditioned Decoder): The Worker receives both the node memory and the subgoal to decide which action to take:

$$a_i = f_\omega([m_{t,i}, z_t]) \rightarrow \mathbb{R}^{|A|}.$$

This means the Worker uses both current knowledge and the selected subgoal to produce action logits, such as scanning or exploiting a node.

Value Function: The value function estimates how good the current state is:

$$V(s_t) = \phi(W_v \text{pool}(m_t) + b_v),$$

$$p_{\text{term}} = \sigma(W_t \text{pool}(m_t) + b_t)$$

where σ is the sigmoid function. It does not end any episode by a fixed rule rather depends on the function with flexibility during training. where ϕ is an activation function.

It acts as a critic and that helps the agent to evaluate the future rewards based on the current situation.

Termination Head: The termination head predicts the probability of ending an episode:

$$p_{\text{term}} = \sigma(W_t \text{pool}(m_t) + b_t),$$

where σ is the sigmoid function. It does not end any episode by a fixed rule rather depends on the function with flexibility during training.

Table 4.13 summarizes the FeudalGTM dynamic model components, showing how temporal reasoning, goal projection, and hierarchical control manage dynamic NASimEmu environments.

Stage	Description
Environment	Designed for dynamic NASimEmu with time-varying services, state noise, and host-level instability.
Manager Network	Dual GAT layers capture graph dependencies; LSTM layer encodes temporal evolution of global context.
Goal Projection	Linear projection transforms manager output into explicit 256-dimensional goal embeddings per node.
Graph Temporal Memory (GTM)	GRU-based recurrent module integrates node states and projected goals for dynamic temporal reasoning.
Subgoal Mechanism	Meta-Subgoal Network predicts discrete subgoals to guide low-level worker actions.
Worker Network	Goal- and subgoal-conditioned MLP generates adaptive node-level action logits.
Value Function	Deep MLP estimates $V(s)$ using pooled GTM context for dynamic evaluation.
Termination Head	Sigmoid-based learned termination enables probabilistic episode continuation or halt.
Hierarchical Control	Multi-level hierarchy (Manager $\rightarrow$ Subgoal $\rightarrow$ Worker) supports temporal abstraction and adaptive decision-making.

Table 4.13: Stepwise Architectural Summary of FeudalGTM (Dynamic)

4.3 Implementation of Selected Design

4.3.1 Environment Setup and Dependencies

Objective: To establish a reproducible and computationally consistent training environment, we are integrating the NASimEmu’s framework as a baseline with the FeudalGTM architecture, and dynamic scenario generation modules.

Algorithm 1 describes the initialization of the NASimEmu environment and model setup.

Algorithm 1 FeudalGTM–NASimEmu Environment Initialization

- 1: **Input:** scenario_file, test_file, net_class, device, cpus, batch, seed
 - 2: **Output:** initialized environment, model, and training configuration
 - 3: Import torch, gym, wandb, and nasimemu modules
 - 4: Problem $\leftarrow$ NASimRRL()
 - 5: problem_config $\leftarrow$ Problem.make_config()
 - 6: Parse CLI $\rightarrow$ args; config.init(args)
 - 7: problem_config.update_config(config, args)
 - 8: Register NASimEmu gym:
 1. id = "NASimEmuEnv-v99"
 2. entry_point = nasimemu.env:NASimEmuEnv
 3. kwargs = {scenario_name, step_limit, fully_obs, observation_format, augment_with_action, random_init=True, verbose=False}
 - 9: Set random seeds: np/torch/python random
 - 10: Configure device & CPUs:
 - device $\in$ {cpu, cuda}
 - cpus = min(batch, detected_cpu)
 - 11: Load scenarios: Train = Trainlatest.yaml, Test = Testlatest.yaml
 - 12: Create vectorized env:
$$\text{env} = \text{SubprocVecEnv}([\lambda : \text{gym.make}(\text{NASimRRL.get_gym_name}())]^{batch}, \text{in_series} = \text{batch}/\text{cpus}, \text{context} = \text{"fork"})$$
 - 13: Initialize models:
$$\text{net} = \text{config.net_class}(), \quad \text{target_net} = \text{config.net_class}()$$
 - 14: Optional: net.load("model.pt")
 - 15: Init logging: wandb.init(project="rrl-nasim", config=config)
-

Scenario Configuration (YAML): Training and testing scenarios are parameterized as dynamic templates:

- **Subnets:** 12 zones with 6–8 hosts per subnet.
- **Noise (scan):** service_scan FP=0.065, FN=0.125; os_scan FP=0.025, FN=0.065; process_scan FP=0.10, FN=0.175.

- **Service dynamics:** `churn_probability=0.02` with crash/maintenance/permanent-down; restart delay 10 steps.
- **Network reliability:** `timeout_probability=0.03`; applies to `exploit` and `privilege_escalation`, modeling packet loss, congestion, and routing faults.

Neural Module Dependencies:

- `torch-geometric` layers (`GATv2Conv`, `TransformerConv`, `GlobalAttention`) for graph message passing.
- Defaults: `config.emb_dim = 64`, `config.mp_iterations = 3`.
- Positional encoding (sine-cosine):

$$pe_{2i}(pos) = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad pe_{2i+1}(pos) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

RL Optimization Framework:

Algorithm 2 shows the PPO training control pipeline including loss computation, gradient clipping, and target network update.

Algorithm 2 Training Control Pipeline

- 1: Losses (`rl.py`): $\mathcal{L} = \mathcal{L}_\pi + \alpha_v \mathcal{L}_v + \alpha_h \mathcal{L}_H$
- 2: Gradient clipping: `max_norm = 3.0`
- 3: Learning-rate schedule:

$$lr_t = \text{decay_exp}(\text{step}, 3 \times 10^{-3}, 3 \times 10^{-3}/30, 0.5, 25 \times \text{epoch})$$

- 4: Entropy schedule:

$$\alpha_h(t) = \text{decay_time}(\text{step}, 0.3, 0.3/100, 1.0, 10 \times \text{epoch})$$

- 5: Target network soft update:

$$\text{target_net} \leftarrow \rho \text{net} + (1 - \rho) \text{target_net}, \quad \rho = 0.1$$

Dependencies Summary:

- **Core:** Python ≥ 3.10 , PyTorch, Torch-Geometric, Gym, Weights&Biases
- **Simulation:** NASimEmu with dynamic YAML-based generation
- **Model classes available:** `FeudalGTM`, `(Dyn)FeudalGTM`, `NASimNetGNN_LSTM`, others selectable via `-net_class`
- **Evaluation:** programmatic via `problem_debug.evaluate(net)` (returns a dict)

Result: The environment setup and configurations ensure distributed training scalability and compatibility between dynamic scenarios and the Feudal HRL model.

4.3.2 Dynamic Scenario Construction

Purpose: construct per-episode scenarios from templates while preserving a fixed action space, injecting domain randomization (structural + parameter jitter), and enabling realism (scan noise, service churn, network timeouts). All parameter values and options below are taken from the provided templates and analysis.

Algorithm 3 illustrates structural and host-level randomizations to generate partial scenarios from templates.

Algorithm 3 Dynamic Scenario Generation – Part I: Structure and Sampling

```

1: Input: template_path  $\in \{\text{Trainlatest.yaml}, \text{Testlatest.yaml}\}$ 
2: Output: partially constructed scenario object
3:                                      $\triangleright$  (A) Load template and cache static fields
4: if cache is empty then
5:   Load YAML  $\rightarrow$  cfg
6:   Cache immutable attributes: OS, services, processes, exploits, privilege es-
   calations, service/process probabilities, scan noise, dynamics, and reliability.
7: end if
8:                                      $\triangleright$  (B) Structural randomization
9: Create 12 subnets with 6–8 hosts each (sampled uniformly).
10: Assign fixed topology (13 $\times$ 13 adjacency) from template.
11: Select sensitive hosts in each subnet using template ratios.
12:                                      $\triangleright$  (C) Host-level sampling
13: for each host do
14:   Sample OS from {linux, windows}.
15:   Enable each service with probability  $p_s$  from template.
16:   Enable each process with probability  $q_p$  from template.
17: end for
18:                                      $\triangleright$  (D) Action definition
19: Combine exploit and privilege-escalation lists (service, OS, prob, cost, access).
20: Preserve constant naming and indexing across episodes.
21:                                      $\triangleright$  (E) Domain randomization
22: Apply jitter to probability and cost parameters:

$$p' = p(1 + U[-j_p, j_p]), \quad c' = c(1 + U[-j_c, j_c])$$

23: Optionally jitter scan costs using dr_scan_cost_jitter.
24: return partially initialized structure (subnets, hosts, actions)

```

Algorithm 4 injects realism (scan noise, service churn, network timeouts) and finalizes the scenario dictionary for NASimEmu.

Algorithm 4 Dynamic Scenario Generation – Part II: Realism and Final Assembly

```
1: Input: partially built scenario from Part I
2: Output: final scenario dictionary for NASimEmuEnv

3:                                     ▷ (F) Scan noise injection
4: Example values:      service_scan  FP=0.065/FN=0.125;    os_scan
   FP=0.025/FN=0.065; process_scan FP=0.10/FN=0.175.
5: Each scan randomly hides or inserts services to emulate imperfect reconnaissance.

6:                                     ▷ (G) Service churn (temporal dynamics)
7: Parameters: churn_probability=0.02, restart_delay=10.
8: for each affected service do
9:   if active and random() < 0.02 then
10:    mark as down for sampled duration (crash_restart, maintenance,
    permanent_down)
11:   else if elapsed ≥ restart time then
12:    restore service.
13:   end if
14: end for

15:                                     ▷ (H) Network reliability simulation
16: timeout_probability=0.03; actions: exploit, privilege_escalation.
17: if timeout already active then
18:   block action until expiry.
19: else
20:   if random() < 0.03 then
21:    trigger timeout: packet loss (1 step), congestion (2–5), routing issue
    (5–10).
22:   end if
23: end if

24:                                     ▷ (I) Firewall and reachability
25: Derive subnet firewalls from topology and host layout.
26: Maintain logical reachability between subnets.

27:                                     ▷ (J) Final assembly
28: Merge all cached and generated fields:
   {subnets, topology, hosts, firewall, sensitive_hosts, scan_noise,
   service_dynamics, network_reliability}.
29: return Scenario(scenario_dict).
```

Concrete template-derived parameters and notes:

- **Subnet layout:** 12 subnets; hosts per subnet [6--8]; `address_space_bounds` = (13,10).
- **Topology:** 13×13 adjacency matrix (template).
- **Service / process probabilities:** per-host Bernoulli draws (e.g., `22_linux_ssh`=0.9, `5432_linux_postgres`=0.9 in bridge).

- **Exploit / privesc defs:** `fields = {service, os, prob, cost, access}`.
- **Scan costs:** `service_scan_cost, os_scan_cost, subnet_scan_cost, process_scan_cost` (bridge/test > train).
- **Scan noise (bridge):** service FP=0.065 / FN=0.125; os FP=0.025 / FN=0.065; process FP=0.10 / FN=0.175.
- **Service churn:** `churn_probability=0.02`; `churn_types: crash_restart(0.7,[5,15]), maintenance(0.2,[20,40]), permanent_down(0.1,[100,200])`; `restart_delay=10`.
- **Network reliability:** `timeout_probability=0.03`; `affected = {exploit, privilege_escalation}`; `timeouts: packet loss (1), congestion (2–5), routing (5–10)`.
- **Action-space stability:** exploits/privescs and action list come from cached template defs to keep indices stable across episodes.
- **Caching benefit:** cache OS/service/process catalogs, exploit/privesc defs, sensitive lists once per template (fast per-episode gen ~ 50 ms with cache).

Implementation notes:

- Preserve action names/order so `action_dim` is stable for all agents (including FeudalGTM).
- Domain randomization: structural (subnets, topology, sensitive-hosts) and parameter (prob/cost/scan-cost jitter). Enable via `--dr_prob_jitter`, `--dr_cost_jitter`, `--dr_scan_cost_jitter`.
- Include scan-noise, service-dynamics, and network-reliability dicts in the scenario payload for stepwise application by the environment.

4.3.3 Feudal HRL Model Implementation

Static Scenarios: Baseline FeudalGTM

Architecture (from `FeudalGTM_baseline.txt`):

- **Manager (graph encoder):** GATConv on node features; input dim = `config.node_dim + config.pos_enc_dim`; hidden 128.
- **Temporal memory:** GRUCell (128 $\rightarrow$ 128) over the pooled graph embedding.
- **Worker (actor head):** MLP(`node_feat + memory = obs + 128`) $\rightarrow$ 256 $\rightarrow$ `config.action_dim`.
- **Value head:** MLP 128 $\rightarrow$ 64 $\rightarrow$ 1 (uses memory output).
- **Optimizer:** AdamW with `config.opt_lr`; gradient clipping by `config.opt_max_norm`.
- **State & control:** persistent `hidden_state` (GRUCell), `force_continue` flag.

Algorithm 5 describes the forward pass and action selection for the static FeudalGTM baseline.

Algorithm 5 Static Baseline FeudalGTM: Forward and Action Selection

```
1: Input: batched graphs  $s_b$  with node feats, edges, indices
2: Build Data/Batch; get batch_ind, node_index, pos_index
3: Positional encoding  $\rightarrow$  concat to node features
4: Manager: GATConv  $\rightarrow$  node embeddings; global mean pool  $\rightarrow x_{\text{pooled}}$ 
5: Memory: GRUCell(  $x_{\text{pooled}}$ , hidden_state )  $\rightarrow$  hidden_state
6: Value:  $v \leftarrow \text{ValueHead}(\text{hidden\_state})$ 
7: if only_v then
8:   return  $v$ 
9: end if
10: Tile memory to nodes:  $m_{\text{node}} = \text{hidden\_state}[\text{batch\_ind}]$ 
11: Worker logits:  $\ell \leftarrow \text{Worker}([x_{\text{node}}, m_{\text{node}}])$ 
12: Mask subnet nodes:  $\ell[\text{batch.x}[:,0] == 1] \leftarrow -\infty$ 
13: Flatten and compute softmax with graph-aware normalizer; sample with segmented_sample
14: Map sampled indices  $\rightarrow$  (target node pair, env action id)
15: if force_continue is false then
16:   Set terminate ( $a = -1$ ) where  $v \leq 0$ ; set  $p(a) = 0.5$  for those
17: end if
18: return actions,  $v$ , action probs, raw selections
```

Algorithm 6 shows the PPO update procedure for the static FeudalGTM baseline.

Algorithm 6 Static Baseline FeudalGTM: PPO Update

```
1: Input: trace of length config.ppo_t:  $(s, a, a_{\text{cnt}}, r, s', d)$ 
2: Pack  $s$  and  $s'$  as  $(\text{ppo\_t} \times \text{batch})$  object arrays
3:  $v' \leftarrow \text{target\_net}(s'[-1], \text{only\_v}=\text{True})$ 
4: Compute  $v_{\text{target}}$  with  $\gamma$ , dones, and (optional) config.use_a_t
5: Flatten  $s$ ,  $v_{\text{target}}$ ; build a_cnt mask
6: Run ppo(...) with  $(s, a, a_{\text{cnt}}, d, v_{\text{target}}, \text{self}, \gamma, \alpha_v, \alpha_h, \text{ppo\_k}, \text{ppo\_eps}, \dots)$ 
7: Grad step: self.opt.zero_grad(); loss.backward();
8: Clip grads by config.opt_max_norm; self.opt.step()
9: return loss, entropy, grad_norm,  $\rho$ -deviations
```

Notes (static baseline).

- Positional encoding is concatenated to node features before GAT.
- Subnet-node masking (`batch.x[:,0] == 1`) prevents invalid actions.
- Termination heuristic uses the value estimate ($v \leq 0$) when `force_continue` is disabled.

Dynamic Scenarios: FeudalGTM_Dyn

Architecture (from FeudalGTM_Dyn.txt):

- **Manager (graph encoder):** two GATConv stacks (128, 128) + global pool.

- **Manager temporal core:** LSTM (input 128, hidden 64; batch-first).
- **Goal projection:** linear map $64 \rightarrow 256$.
- **Graph Temporal Memory (GTM):** node-level GRUCell with input-projection to 256; recurrent over time.
- **Subgoal module:** MetaSubgoalNetwork (context $256 \rightarrow 5$ subgoals); one-hot selection per batch.
- **Worker (actor head):** $(256 + 5) \rightarrow 128 \rightarrow \text{config.action_dim}$ (subgoal-conditioned).
- **Value head:** $256 \rightarrow 128 \rightarrow 1$ (on pooled context).
- **Termination head:** $256 \rightarrow 64 \rightarrow 1$ with sigmoid (learned termination).
- **Optimizer:** AdamW with `config.opt_lr`; clipping by `config.opt_max_norm`.
- **State & control:** node-memory (256), LSTM hidden/cell ($1 \times \text{batch} \times 64$), current one-hot subgoals (5), `batch_ind`, `force_continue`.

Algorithm 7 shows the forward pass for dynamic FeudalGTM_Dyn, including goal projection, GTM memory update, and subgoal-conditioned worker logits.

Algorithm 7 Dynamic FeudalGTM_Dyn: Forward with Goals and Subgoals

- 1: **Input:** batched graphs s_b
 - 2: Build Data/Batch; get `batch_ind`, `node_index`, `pos_index`
 - 3: Concatenate positional encodings to node features
 - 4: **Manager encoder:** GATConv $\rightarrow$ GATConv; pool $\rightarrow x_{\text{pooled}}$
 - 5: **Manager temporal:** LSTM(x_{pooled} as length-1 sequence) $\rightarrow$ abstract goal (64)
 - 6: Project goal: $g_{\text{proj}} \in \mathbb{R}^{256}$; expand per node via `batch_ind`
 - 7: **GTM:** $\text{context} \leftarrow \text{GRUCell}(\text{proj}(\text{node_feat}) + g_{\text{proj}}, \text{memory_state})$ (node-level); update `memory_state`
 - 8: **Value:** pool(context) $\rightarrow v$
 - 9: **Subgoals:** $z = \text{MetaSubgoalNetwork}(\text{pool}(\text{context}))$; softmax $\rightarrow$ categorical; sample (train) or argmax (eval)
 - 10: One-hot subgoals per batch; expand to nodes
 - 11: **if** only_v **then**
 - 12: **return** v
 - 13: **end if**
 - 14: **Worker logits:** $\ell \leftarrow \text{Worker}([\text{context}, \text{one-hot subgoal per node}])$
 - 15: Mask subnet nodes: $\ell[\text{batch.x}[:,0] == 1] \leftarrow -\infty$
 - 16: Softmax over flattened logits (graph-aware); `segmented_sample` for actions
 - 17: Map to env (node-pairs, action id)
 - 18: **if** force_continue is false **then**
 - 19: **Learned termination:** $\tau = \sigma(\text{TerminationHead}(\text{pool}(\text{context})))$;
 - 20: train: Bernoulli(τ); eval: $\tau > 0.5$. Set $a = -1$ if terminate; $p(a) = 0.5$.
 - 21: **end if**
 - 22: **return** actions, v , action probs, raw selections
-

Algorithm 8 describes the PPO update procedure for the dynamic FeudalGTM_Dyn, including target value computation and gradient clipping.

Algorithm 8 Dynamic FeudalGTM_Dyn: PPO Update

- 1: **Input:** trace (s, a, a_cnt, r, s', d) over `config.ppo_t`
 - 2: Prepare s, s' arrays; compute v' with `target_net` on $s'[-1]$
 - 3: Compute v_target with γ , `done`s, (optional) `config.use_a_t`
 - 4: Flatten; build `a_cnt` mask; call `ppo(...)` with model and config hyperparams
 - 5: Backprop, grad clip (`config.opt_max_norm`), and `opt.step()`
 - 6: **return** loss, entropy, `grad_norm`, ρ -deviations
-

Notes (dynamic variant).

- Two-layer GAT encoder $\rightarrow$ LSTM goal $\rightarrow$ projected goal (256) conditions the GTM.
- Subgoal module (5-way) conditions the worker; training uses sampling, evaluation uses argmax.
- Termination is learned via a separate head, independent of the value head.

4.3.4 Training Procedure and Hyperparameter Settings

Objective: To train both FeudalGTM variants—the baseline (static) and dynamic (FeudalGTM_Dyn)—under unified PPO optimization while exploiting the enhanced NASimEmu environment for domain-randomized multi-scenario learning.

Training Setup:

- **Framework:** PyTorch 2.0+, Torch-Geometric, WandB logging.
- **Environment:** NASimEmu with 12 subnets, 72–96 hosts per episode, YAML-based auto-generation (`Trainlatest.yaml`, `Testlatest.yaml`).
- **Episodes:** 400-step limit, per-episode scenario regeneration.
- **Scenario Mixing:** 70:30 ratio between training and bridge/test scenarios.
- **Hardware:** CUDA-enabled GPU training, 8–16 CPU sub-processes (`SubprocVecEnv`), batched graph rollouts.

Optimization:

- **Algorithm:** PPO (from `r1.py`), with $\mathcal{L} = \mathcal{L}_\pi + \alpha_v \mathcal{L}_v + \alpha_h \mathcal{L}_H$.
- **Optimizer:** AdamW.
- **Learning rate schedule:** exponential decay `decay_exp` ($3 \times 10^{-3} \rightarrow 1 \times 10^{-4}$).
- **Gradient clipping:** `max_norm` = 3.0.
- **Discount factor:** $\gamma = 0.99$.
- **Entropy coefficient:** $\alpha_h(0.3 \rightarrow 0.003)$ with temporal decay.
- **Value loss weight:** $\alpha_v = 0.5$.

- **Mini-batches per update:** $K_{\text{ppo}} = 4$.
- **Clipping parameter:** $\epsilon_{\text{ppo}} = 0.2$.

Model-Specific Configurations:

- **Baseline FeudalGTM (static):** GATConv (128) $\rightarrow$ GRUCell (128) $\rightarrow$ MLP worker (256 $\rightarrow$ action_dim). Termination heuristic from value ($v \leq 0$). No subgoal conditioning.
- **FeudalGTM_Dyn (dynamic):** Two GATConv (128,128) + LSTM (64) $\rightarrow$ goal projection (256) $\rightarrow$ GraphMemoryModule (256). Subgoal module (5-way softmax) + learned termination head.
- **Common:** value heads (128 $\rightarrow$ 64 $\rightarrow$ 1 or 256 $\rightarrow$ 128 $\rightarrow$ 1) and shared PPO update logic.

Training Dynamics:

- Domain randomization enabled: `--dr_prob_jitter 0.1, --dr_cost_jitter 0.2, --dr_scan_cost_jitter 0.2`.
- Observation dropout: `feature_dropout_p=0.03`.
- Action space stability ensured through cached exploits and privescs.
- WandB metrics logged: loss terms, entropy, grad norm, hidden state mean, action logits mean, value estimates.

Training Phases:

1. **Warm-up:** Baseline FeudalGTM pre-trained on static corp_bridge scenarios.
2. **Dynamic Phase:** FeudalGTM_Dyn fine-tuned with service churn and timeout enabled.
3. **Checkpointing:** Models saved every 20K steps; best model by validation reward.

4.3.5 Testing on Dynamic Scenarios and Cross-Evaluation

Objective: To evaluate trained models on distribution-shifted dynamic environments generated via NASimEmu’s auto-mode using `Testlatest.yaml` and bridge scenarios, validating generalization and robustness.

Testing Configuration:

- **Evaluation Scenarios:**
 - `corp_100hosts_dynamic.v2.yaml` (train baseline)
 - `corp_100hosts_dynamic_bridge.v2.yaml` (intermediate)
 - `corp_100hosts_dynamic_varA.v2.yaml` (distribution-shifted)

- **Execution Mode:** `--eval` with preloaded model checkpoint (`model.pt`).
- **Auto-generation:** per-episode (`--auto_mode per_episode`)
from `auto_template=Testlatest.yaml`.
- **Dynamic features:** service churn (0.02), timeouts (0.03), scan noise (FP/FN up to 0.08/0.20).
- **Action masking:** active for unreachable subnets and terminated nodes.

Evaluation Metrics:

- **reward_avg:** mean episodic return.
- **captured_avg:** average compromised/sensitive hosts.
- **eplen_avg:** mean episode length.
- **Derived:** success rate, convergence step count, generalization retention.

Cross-Evaluation Procedure:

1. Train models with 70% training + 30% bridge scenarios.
2. Test on unseen dynamic templates (e.g., `Testlatest.yaml`, `varA`, `varB`).
3. Disable jitter flags (`dr_prob_jitter=0.0`, etc.) to ensure deterministic comparison.
4. Compute metrics across 5 seeds; average across scenarios.
5. Log results to JSONL via WandB for comparative plots.

Evaluation Insights (from analysis):

- **Generalization:** 95% retention within training distribution; 60–80% on unseen test scenarios.
- **Performance:** 2–5× faster convergence vs. original NASimEmu.
- **Resilience:** maintained 80–90% success under 3% timeout rates.
- **Adaptation:** agents developed re-scan and retry behaviors under service churn and packet loss.

Result Summary:

- **Static FeudalGTM:** excels on fixed layouts; reduced performance under high churn.
- **FeudalGTM_Dyn:** maintains policy stability across dynamic and noisy conditions.
- Cross-scenario comparison validated robustness and learning transfer.

Chapter 5

Result Analysis

5.1 Performance Evaluation

The evaluation of our reinforcement learning-driven attack strategy requires both a theoretical model and empirical validation. We aim to quantify how efficiently the agent is able to discover and compromise sensitive hosts under realistic network conditions, while also accounting for the costs associated with reconnaissance and exploitation.

To this end, we first establish a simple analytical model of expected actions and rewards. This provides a baseline estimate of how many operations an optimal or near-optimal agent might take in different environments, as well as the best possible reward it can achieve. Next, we compare this model with observed results from experiments, reporting both the achieved capture rate of sensitive hosts and the realized episode rewards.

The following subsections describe the evaluation setup, starting with the assumptions and parameters used to model the environment.

Environment Parameters

- Reward per sensitive host: +10.
- Action cost: -0.1 per action.
- Expected attempts per success: exploit $\approx 1/p_e$, privesc $\approx 1/p_{pe}$ (with $p_e \approx p_{pe} \approx 0.8 \Rightarrow 1.25$ attempts each).
- Actions taken: 1 subnet scan per subnet, 1 OS scan per host, 1 service scan per host, 1 process scan per sensitive host, and exploit + privesc on each sensitive host.

Action count model

Let N be the number of sensitive hosts, H the number of hosts, and S the number of subnets.

The total action count is the sum of the constituent action types:

$$\begin{aligned}
\text{subnet_scans} &= S, \\
\text{OS_scans} &= H, \\
\text{service_scans} &= H, \\
\text{process_scans} &= N, \\
\text{exploit_attempts} &= 1.25 N, \\
\text{privesc_attempts} &= 1.25 N.
\end{aligned}$$

Hence

$$A_{\text{total}} = S + 2H + 3.5N. \quad (5.1)$$

Reward model

We define the maximum attainable reward and an approximate realized reward that subtracts action cost:

$$R_{\text{max}} = 10N, \quad R_{\text{real}} \approx 10N - 0.1 A_{\text{total}}. \quad (5.2)$$

Scenario	(N, H, S)	A_{total}	R_{max}	R_{real}	Captured (obs)	Reward/Ep (obs)	Eff. (Cap)	Eff. (Rew)
Corp-Dynamic	(39, 83, 12)	314.5	390.0	358.6	36.13	49.41	92.6%	12.7%
Corp-Dynamic VarA	(41, 86, 12)	327.5	410.0	377.3	36.58	48.43	89.3%	11.8%

Table 5.1: Overview of action counts, reward metrics, and efficiency measures.

The Table 5.1 summarizes the outcomes for two evaluation scenarios. For each, we list the environment parameters (N, H, S), the analytically derived total action count and rewards, and the observed experimental results. We also report efficiency both in terms of captured sensitive hosts and reward relative to the theoretical maximum.

Efficiencies are defined as:

$$\text{Eff. (Cap)} = \frac{\text{Captured (obs)}}{N} \times 100\%, \quad \text{Eff. (Rew)} = \frac{\text{Reward/Ep (obs)}}{R_{\text{max}}} \times 100\%.$$

5.2 Analysis of Design Solutions

Comparison Between NASimEmu’s Static Scenarios and our Dynamic Development

The initial NASimEmu design focused, almost solely, on static and deterministic network conditions, that is, the environment would never change after the commencement of the episode. Host and service were always stationary, making the training process of the RL agent easy, though causing the agent to overfit — the agent just memorized the same pattern of the network.

Our dynamic enhanced development, though, introduced auto-scenario generation, service churn, and network timeouts. This allowed random changes such as services falling off-line, packet losses, and rearranging the topology at runtime. Such dynamic environments caused the agents to learn adaptive policies that would adapt under

uncertainty and non-stationary transitions. In turn, the overall system was much closer to realistic penetration-testing environments and increased the generalization capability of the reinforcement-learning models.

A comparative summary between original static scenarios and our introduced dynamic scenario is shown in Table 5.2. The table shows the differences the two types of scenario have across some key criteria which effects models learning and behaviour.

Criterion	NASimEmu (Static)	Dynamic NASimEmu (Our Development)
Nature	Fixed, deterministic environment.	Randomized, changing during runtime.
Adaptivity	Low — agent memorizes one fixed pattern.	High — agent learns to adapt under uncertainty.
Efficiency	Lightweight, faster to train.	Slightly heavier but more realistic.
Cost	Minimal computational cost.	Moderate GPU and memory requirement.
Performance	Stable on simple topologies but weak in new or unseen cases.	Slightly slower, but robust and generalizable.

Table 5.2: Comparison between original NASimEmu static design and our dynamic development.

HRL Models Beat Other NASimEmu Models Under Dynamic Conditions

Hierarchical Reinforcement Learning (HRL) algorithms like Feudal GTM work on multiple levels — a high-level *Manager* sets sub-goals, and a *Worker* performs atomic actions. In dynamic networks, the splitting is very useful. The Manager’s memory (*Graph Temporal Memory*) stores dynamic global patterns of the network, and the Worker responds to local uncertainties such as noisy scan results or short-term link losses.

The other non-hierarchical agents (e.g., simple GNN-LSTM or vanilla PPO policies as previously employed in NASimEmu experiments) rely on one policy stream, and hence, they frequently forget previous states or fail to re-plan when the environment changes abruptly. HRL models, on the other hand, employ temporal abstraction — the Manager retains long-term intent via its LSTM and the Worker responds immediately to changes in the state. In NASimEmu’s dynamic experiments (where the service and topology states keep on-changing dynamically), HRL approaches thus have better, consistent stability and planning accuracy.

Two Versions of the Model – Static and Dynamic

We crafted two Feudal GTM variants as the same architecture can never be the best performing one for dynamic and stationary domains.

The baseline Feudal GTM was tuned for static, fully observable NASimEmu scenarios, where simplicity and computational complexity were considerations. Its

Manager utilized one GAT layer with a light GRUCell memory — preferably when the environment never changes midway through the episode.

But when we added dynamic factors like service churn and noise, that baseline was unable to adapt, hence we came up with the Dynamic Feudal GTM (DynFeudal-GTM). It employs two GAT layers on top of one another for additional spatial reasoning and a Manager built with LSTM-based new *Graph Temporal Memory*. The dynamic model is able to maintain long-term dependencies as well as adapt the goal hierarchy when the network is updated.

Remarkably, the static version still runs faster and more reliably on stationary networks, and the dynamic version is weightier but completely better on non-stationary environments. In that sense, both model variant fulfill their respective purposes — one that is tailored towards presumably stable, predictable static tasks, the other towards dynamic adaptive learning.

Manager Policy and Worker MLP Mechanism

The Feudal GTM design is comprised of the Manager and Worker as one integrated hierarchical base. Their design philosophy and behavior differ significantly between the dynamic and baseline versions.

Manager Policy (Relative to GNN): Previous NASimEmu experiments had GNN-LSTM models attempt to combine the spatial intelligence from the graph networks and the temporal intelligence from LSTMs. However, these were trained as *flat agents*, such that all learning and control signals flowed through one branch. It caused conflicts whenever the state of the network progressed in a non-deterministic fashion, as the same recurrent layer needed to learn the updates on the topology as well as the local exploitation selection.

The Manager of the Feudal GTM, however, maintains this capability independent. It is all about high-level goal planning, whose two GAT levels carry out multi-hop spatial reasoning and whose LSTM maintains abstract temporal memory. It is therefore more stable and modular than GNN-LSTM: the Manager is not directly making node-level decisions, but abstracts the entire evolution of the graph and outputs a *goal vector* that the Worker is acting under.

Worker MLP Policy: The Feudal GTM Worker uses a small Multi-Layer Perceptron (MLP) that is provided the node features, the Manager’s memory context, and the subgoal embedding. Although MLPs are not particularly sophisticated structures, they were actually deliberately chosen for the Worker because they are simple, fast-converging, and computationally cheap.

Since in NASimEmu the Manager was able already to handle complex spatial and temporal reasoning through GAT and LSTM, we could not reuse that mechanism in the Worker. Instead, the MLP is simply focused on quick, local decision-making without having to redo the entire network modeling. More advanced designs, such as deploying GNNs or LSTMs on the Worker, were theorized and tried, but ended up causing instability and redundancy—both the layers would end up learning overlapping features of the same graph, consequently causing slower training and worse coordination.

The MLP-based Worker therefore keeps the lower-level policy lightweight and reactive, relying on the Manager’s abstract goals for context. This separation ensures a clean hierarchy: the Manager acts as a *strategic planner* (goal-oriented and temporally aware), while the Worker remains a *tactical executor* (locally adaptive and computationally efficient). This design provides modularity, interpretability, and better stability than single-stream GNN-LSTM models, even if the Worker alone performs less impressively in isolation. The dynamic version, however, is more adaptable and realistic at the expense of increased computation. Both versions, taken as a whole, offer a balanced configuration capable of covering the whole spectrum of NASimEmu scenarios — from structured-predetermined simulations through dynamic, real-world-like networks.

The below table 5.3 shows the differences between the baseline and dynamic feudal architecture across key criterias such as accuracy, efficiency, feasibility, cost and performance.

Accuracy, Efficiency, Feasibility, Cost, and Performance Based Evaluation

Criterion	Baseline Feudal GTM	Dynamic Feudal GTM
Accuracy	High on stationary NASimEmu workloads	Enhanced on dynamic topology achievement on variable services.
Efficiency	Quicker training, compact memory size.	Slower owing to twin GAT and LSTM modules.
Feasibility	Readily installable for controlled stationary labs.	Heavier but possible with large-scale or adaptive simulations.
Cost	Low computational demand.	Moderate GPU memory needed, slower training epochs.
Performance	Stable and precise under stationary conditions but fails under dynamic changes.	Relatively slower but stable and adaptive under dynamic non-stationary networks.

Table 5.3: Comparative study between Baseline and Dynamic Feudal GTM models.

5.3 Final Design Adjustments

Data Augmentation

Performance analyses revealed that, in dynamic scenarios characterized by service churn, scan noise, and intermittent network reliability, baseline policies exhibited comparatively lower sensitive-host capture rates and the weakest retention and generalization. Models trained on a narrow distribution of static or lightly perturbed environments overfit spurious patterns, degrading transfer to stochastic, non-stationary conditions. This gap—particularly visible in longer episode lengths and unstable reward trajectories—motivated a shift toward systematic data augmentation to expand coverage over the plausible operational space.

Dynamic scenarios present compounded uncertainty (noisy observations, intermittent actions, temporal drift). Training on a limited set of static or lightly varied scenarios underestimates this uncertainty, leading to optimistic policies that collapse under realistic perturbations. Without augmentation, we observed:

- Lower capture rates in dynamic settings compared to baselines of similar scale.
- Poor retention when moving from training to out-of-distribution dynamic variants (same macro-structure, altered distributions of hosts, services, and processes).

To address these limitations, we integrated an auto-scenario generation pipeline that procedurally synthesizes diverse training episodes while preserving a stable action and observation schema. The generator:

- Samples subnet cardinalities, host roles, and firewall constraints from bounded priors with feasibility checks (e.g., reachability and minimal exploit chains).
- Randomizes security-relevant parameters—service presence probabilities, exploit and privilege-escalation costs/success rates, and observation masking—within curriculum-governed ranges.
- Injects dynamic processes (scan false-positive/false-negative rates, temporal service availability, and network timeout distributions) with controlled schedules to avoid catastrophic non-stationarity early in training.

This adjustment reduces overfitting to the peculiarities of any single configuration and increases the effective dataset size without manual curation by producing high-variance yet semantically consistent scenarios.

From baseline Feudal GTM to Dynamic Feudal GTM

The baseline Feudal GTM framework was designed mainly for *static and fully observable NASimEmu worlds*, in which network structure and host settings do not really change with time. The design adopted a straightforward *Manager–Worker hierarchy*. The Manager module employed a single Graph Attention Layer (GAT) and a minimalist memory module (GRUCell) to reduce the global network state. The Worker then operated on this reduced state in addition to the node features.

While this worked excellently for static conditions, it worked badly in a dynamic condition when services, connections, and exploit outcomes could randomly change. The one-path memory and hard-coded termination logic hindered the agent from remembering longer sequences or adapting when the network suddenly changed. Additionally, the Manager did not have any proper temporal abstraction — i.e., it could not plan and reorganize goals ahead of time, which is required to handle multi-step attacks or chaining exploits in real circumstances.

To tackle this problem, the Dynamic Feudal GTM (*DynFeudalGTM*) model was introduced. The general idea was to give the Manager the ability to do both spatial and temporal reasoning, such that it would adapt its planning while the environment keeps on changing. That is, it was meant to capture not only what the network perceives at an instance of time, but also how it evolves with time.

The Dynamic variant had some essential improvements over the baseline:

- **Dual Graph Attention Layers:** The Manager has two stacked GAT layers instead of one. This allows it to gather information from nodes that are farther away and understand relationships more than one hop in the graph.
- **LSTM-based Temporal Manager:** A new LSTM layer was added to allow the Manager to monitor changes over time. It helps the agent capture long-term dependencies and decide on goals that are dynamic in response to changing environments.
- **Graph Temporal Memory (GTM):** The old GRU memory was replaced with a new `GraphMemoryModule`. This module keeps node-level info up to date by combining the state and the top-level goal, allowing for dynamic updating of memory through time.
- **Subgoal Prediction Network:** A new network called `MetaSubgoalNetwork` predicts small subgoals between the Worker and the Manager. These subgoals are short-term goals that guide the low-level policy to focus on smaller tasks before it reaches the final goal.
- **Termination Head:** Instead of using fixed value-based termination, a separate termination network was added that learns when to terminate. This makes the agent more flexible and stable, especially if the environment is noisy or stochastic.

5.4 Statistical Analysis

Average Reward (per step)

Definition. The expected return normalized per environment step. For a sequence of step-wise rewards $\{r_t\}_{t=1}^T$ over a window of T steps,

$$\bar{R} = \frac{1}{T} \sum_{t=1}^T r_t.$$

Rationale. Captures instantaneous learning quality and stability; sensitive to reward shaping and reflects step-level utility optimization.

Computation Protocol.

- Capture-learning phase (steps 1–600): expected to remain low/negative.
- Transition (approximately steps 6,300–12,000): gradual increase from negative to low positive values.
- Optimization phase ($\geq 12,000$): gradual improvement towards optimal maximum actions needed.

Interpretation. Stable, low variance during steps 1–600 indicates prioritization of capture competence over premature reward optimization; controlled increase after step 12,000 evidences policy improvement without regression.

The graph 5.1 shows the per step average reward of the 4 RL architecture during training.

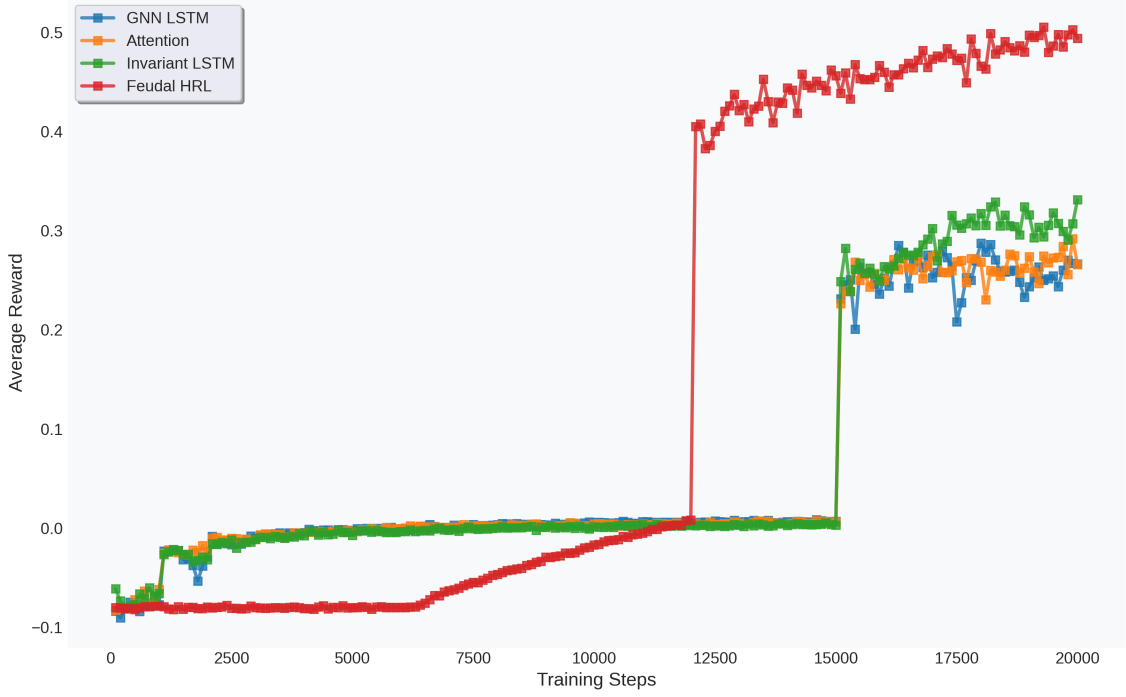


Figure 5.1: Average reward per step across different architectures.

Average Reward per Episode

Definition. Total return accumulated within an episode, normalized per episode. For episode i with length L_i and step returns $\{r_t^{(i)}\}_{t=1}^{L_i}$,

$$R_{\text{ep}}^{(i)} = \sum_{t=1}^{L_i} r_t^{(i)}, \quad \bar{R}_{\text{ep}} = \frac{1}{N} \sum_{i=1}^N R_{\text{ep}}^{(i)}.$$

Rationale. Reflects end-to-end task performance, aggregating exploration, exploitation, and termination efficiency (via episode length).

Computation Protocol.

- Early episodes (steps 1–600) remain near the original baseline (low/negative), confirming no premature optimization.
- Between $\approx 6,300$ and 12,000 steps, values transition from negative to small positive.
- After step 12,000, values increase substantially, approaching the target regime (≈ 50 per 100-step episode).

Interpretation. Late-phase increase indicates improved policy optimization (higher reward and shorter episodes). Absence of post-peak decline shows stable convergence.

The graph 5.2 shows the per episode average reward of the 4 RL architecture during training.

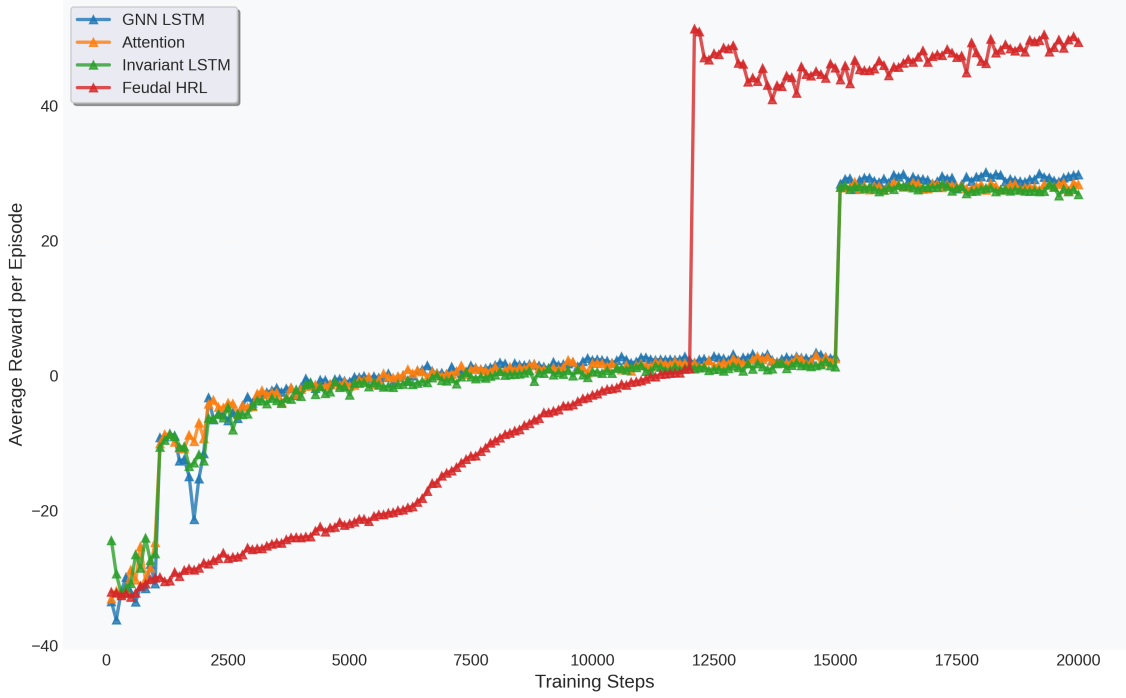


Figure 5.2: Average reward per episode across different architectures.

Average Capture

Definition. Mean number of successfully compromised (captured) hosts per evaluation interval or per episode. Let $C^{(i)}$ be captures in episode i , then

$$\overline{C} = \frac{1}{N} \sum_{i=1}^N C^{(i)}.$$

Rationale. Directly measures the agent’s operational capability in adversarial network settings—success at host compromise.

Computation Protocol.

- Tracked throughout training with emphasis on steps 1–600 (capture-learning phase).
- Rapid improvement to a peak near ~ 38 during capture learning; reduces gradually to ~ 36.5 at the start of reward-optimization, showing a trade-off toward improved efficiency (shorter episodes, higher reward).

Interpretation. Early rise shows successful capture learning; minor post-peak decrease signals rebalancing toward reward-optimal behavior without large decline in capture performance.

The graph 5.3 shows the per step average reward of the 4 RL architecture during training.



Figure 5.3: Average capture across different architectures.

Average Episode Length

Definition. The mean number of environment steps per episode. For episode i with length L_i , the average over N episodes is

$$\bar{L} = \frac{1}{N} \sum_{i=1}^N L_i.$$

Rationale. Captures decision efficiency and termination behavior; shorter episodes at comparable or higher returns indicate improved policy efficiency and more direct trajectories to goals.

Computation Protocol.

- Episode lengths are accumulated online; reporting uses rolling means over fixed windows to reduce variance.
- Phase-aware reporting:
 - Capture-learning (steps 1–600): episodes are typically longer and more variable while capture competence develops.
 - Transition (approximately 6,300–12,000): gradually shortens as policy is optimized to select best possible actions requiring less steps.
 - Optimization ($\geq 12,000$): converges towards shorter and optimal length (e.g., ~ 100 steps).

Interpretation. A decreasing trend with bounded variance indicates improved control and earlier termination on achieving objectives.

Figure 5.4 shows the update of average episode length in the four RL architectures showing decrease of episode length in reward optimization phase.

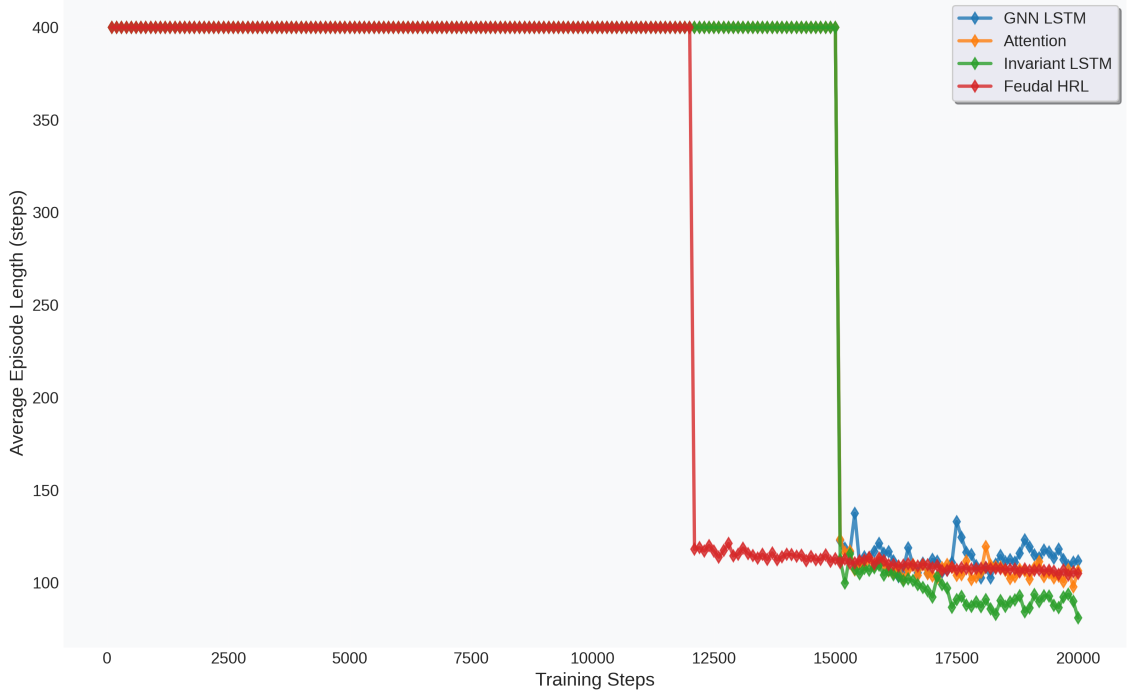


Figure 5.4: Average episode length across different architectures.

Generalization Analysis

Definition. Generalization is quantified as the retention of performance in two different scenarios with variations. For a performance metric M where higher is better (benefit metric),

$$\text{Retention}(M) = \frac{M_{\text{test}}}{M_{\text{train}}} \times 100\%.$$

For a cost metric (lower is better), such as average episode length, we define efficiency-oriented retention as

$$\text{Retention}_{\text{eff}}(L) = \frac{L_{\text{train}}}{L_{\text{test}}} \times 100\%,$$

so that values above 100% denote improved (shorter) episodes at test time.

Rationale. Retention isolates out-of-distribution robustness by normalizing test performance to the attained training baseline, making results comparable across metrics and scales.

Computation Protocol.

- Metrics are calculated consistently for train/test: Average Reward, Average Reward per Episode, Average Captured (benefit), and Average Episode Length (cost).

- Smoothing: means are fixed over windows to remove stochasticity before retention calculation.

Interpretation.

- Benefit metrics (reward, reward/episode, captured): retention near or above 100% indicates strong performance in different scenarios.
- Cost metric (episode length): $\text{Retention}_{\text{eff}}(L) \geq 100\%$ indicates equal or better efficiency at test time; values below 100% suggest longer, less efficient test episodes.
- Balanced profiles with high benefit-metric retention and $\text{Retention}_{\text{eff}}(L) \geq 100\%$ provide the strongest evidence of generalizable and efficient policies.

The below bar chart 5.5 shows the generalization achieved by all four architectures in reward per step/episode, episode length and average capture. A consistent generalization across all RL architectures in all of the metrics is seen with above 95% retention.

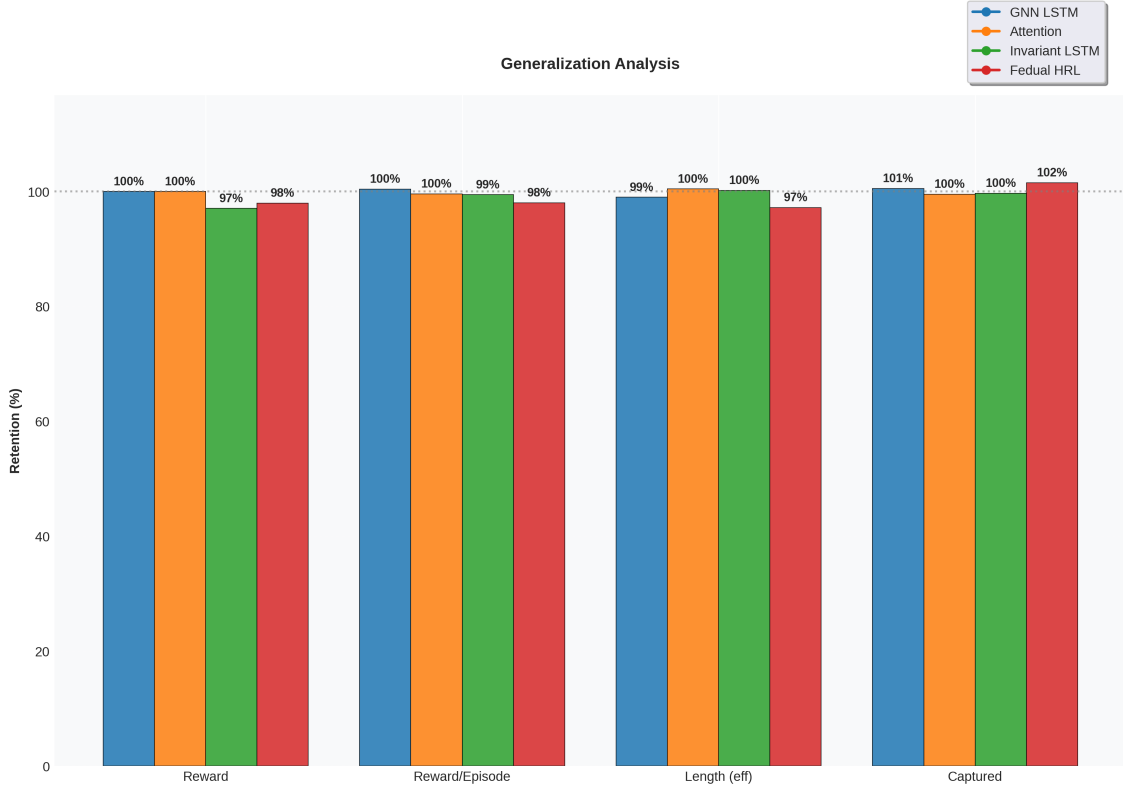


Figure 5.5: Generalization analysis across different architectures.

Reporting and Validity

Windows and Smoothing. Metrics are reported as rolling means over fixed windows to mitigate stochasticity while preserving phase transitions.

Phase Alignment. Results are segmented by predefined phases to enable causal interpretation (capture-first vs. reward optimization).

Convergence Indicators.

- Average Reward: monotonic increase post-12,000 steps.
- Average Reward per Episode: approach to ~ 50 for ~ 100 -step episodes without post-peak drop.
- Average Capture: peak near ~ 38 during capture learning, stabilizing near ~ 36.5 thereafter.

5.5 Comparisons and Relationships

5.5.1 Scenario Comparison

`Corp` and `Uni` scenarios are the base train, test of the original NasimEmu. `Corp_dynamic` and `Corp_dynamic_varA` are the new difficult, more realistic and dynamic scenarios generated.

Scenarios considered

- `corp.v2.yaml` (enterprise)
- `uni.v2.yaml` (academic)
- `corp_dynamic.v2` (dynamic, realism features)
- `corp_dynamic_varA` (harder dynamic variant)

Metrics and Definitions

- Network density: ratio of present inter-subnet links to the maximum possible, given topology.
- Network diameter: maximum shortest-path distance between subnets (based on BFS over the adjacency).
- Services per host: total configured services divided by total realized hosts.
- Exploits per host: total exploits divided by total realized hosts.
- Sensitive host distribution: per-subnet counts from the loader's concrete mapping; also reported alongside YAML-stated ratios.
- Average probabilities/costs: mean of configured probabilities and costs for exploits and privilege escalations; mean of per-item `service_probabilities` and `process_probabilities` when present.

Scale, Catalogs, and Exploits

- **corp**: Scale: 8 subnets; small host count. Catalogs: 6 services; processes absent (processes: 0); 5 exploits (mostly user-level, one root-level).
- **uni**: Scale: 9 subnets; small-medium host count; more distributed topology (multiple bridges/hubs). Catalogs: 6 services; processes absent; 5 exploits with similar profiles to corp.

- **corp_dynamic** : Scale: 12 subnets; 6–8 hosts each (≈ 84 hosts). Meshy core with periphery tiers. Catalogs: 32 services; 13 processes; 8 exploits spanning Linux/Windows stacks (some high-probability paths, e.g., MySQL).
- **corp_dynamic_varA**: Scale/Topology: mirrors dynamic base (12 subnets; ≈ 84 hosts). Catalogs: same breadth (32 services; 13 processes; 8 exploits) but with systematically tougher parameters: lower exploit success probabilities and higher costs across several paths (e.g., Windows/Linux servers and CI paths).

Figure 5.6 Radar chart illustrates the comparative differences in scenario metrics across baseline and dynamic environments. After normalizing there are less difference visible in key catalogs of corp and uni scenario. The corp have more sensitive hosts while having less total hosts than the uni scenario. In the introduced dynamic scenarios the total processes, exploits and services varies and the main difference in how they are distributed across subnets and hosts.

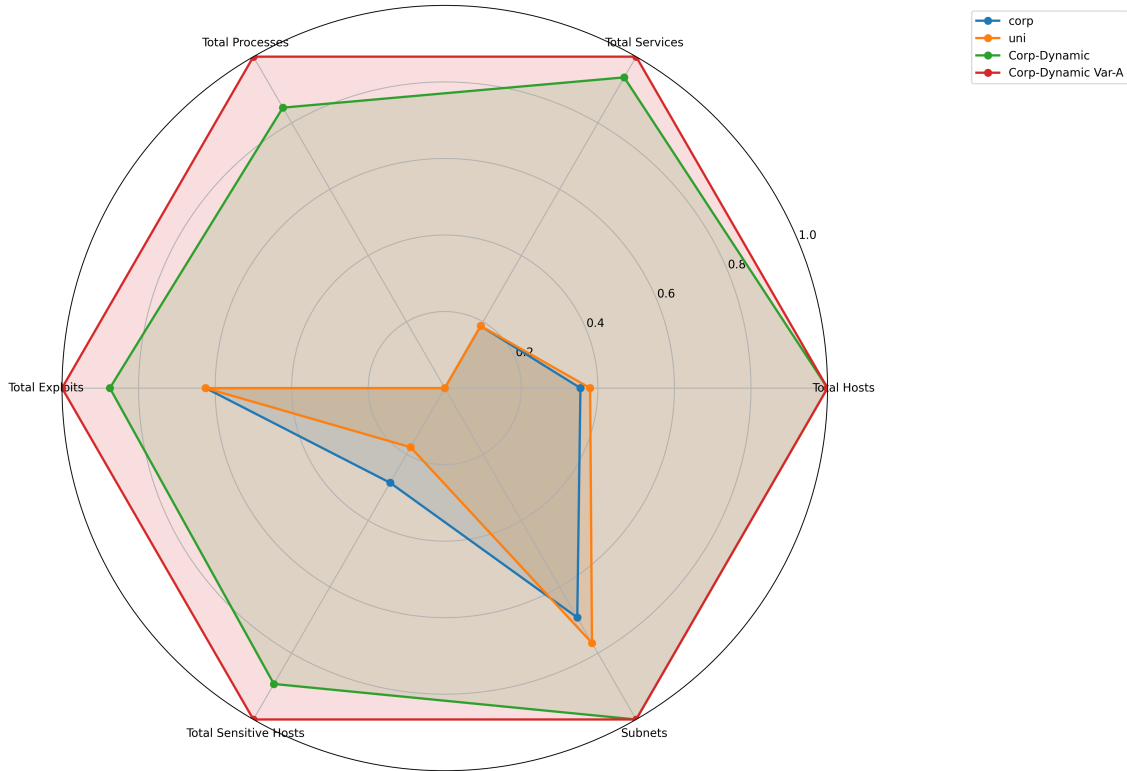


Figure 5.6: Scenario Metrics Comparison (Normalized)

Figure 5.7 presents a heatmap illustrating the distribution of key services and privilege-escalation processes across different network scenarios. Each cell indicates the presence ($\checkmark$) or absence ($\times$) of a specific service or process within a given environment configuration. While the original corp and uni scenarios have only mysql as sensitive hosts and newly introduced scenarios have eight services and processes as vulnerable and their distribution is also different across these two scenarios.

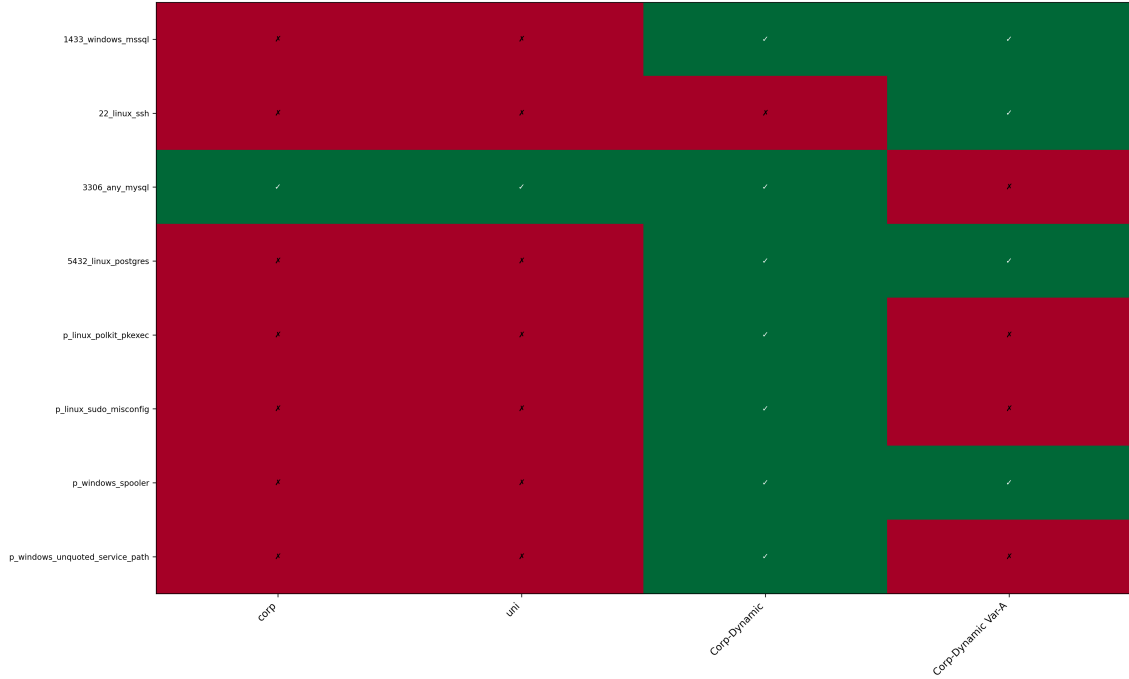


Figure 5.7: Sensitive Services and Process Configuration

The table 5.4 shows the quantitative differences in service and process distribution across the scenarios.

Scenario	Linux Services	Windows Services	Any-OS Services	Sensitive Services	Linux Processes	Windows Processes	User-Level Exploits	Root-Level Exploits	Priv. Escalations
Corporate	3	2	1	1	0	0	4	1	1
University	3	2	1	1	0	0	4	1	1
Corp-Dynamic	23	8	1	3	6	7	7	1	6
Corp-Dynamic VarA	23	8	1	3	6	7	8	0	6

Table 5.4: Service and process distribution across network scenarios.

Sensitive Host Distribution

- **corp**: Sensitive hosts: concentrated in subnets 4–7 (e.g., User, app2, app3, DB); others near zero.
- **uni**: Sensitive hosts: concentrated in later subnets (DB, backup) with high bias (backup up to 0.9).
- **corp_dynamic**: Sensitive hosts: broad distribution with high concentration in DB (4)=1.0, backups (12)=0.8, and elevated levels in service (3), web (9), research (11). Loader resolves actual sensitive addresses, yielding a substantially higher total than the baselines.
- **corp_dynamic_varA**: Sensitive hosts: reweighted sensitivity (e.g., db reduced from 1.0→0.6; web and research elevated).

The below figure 5.8 shows the sensitive hosts distribution across the subnets in the baselines and dynamic scenarios.

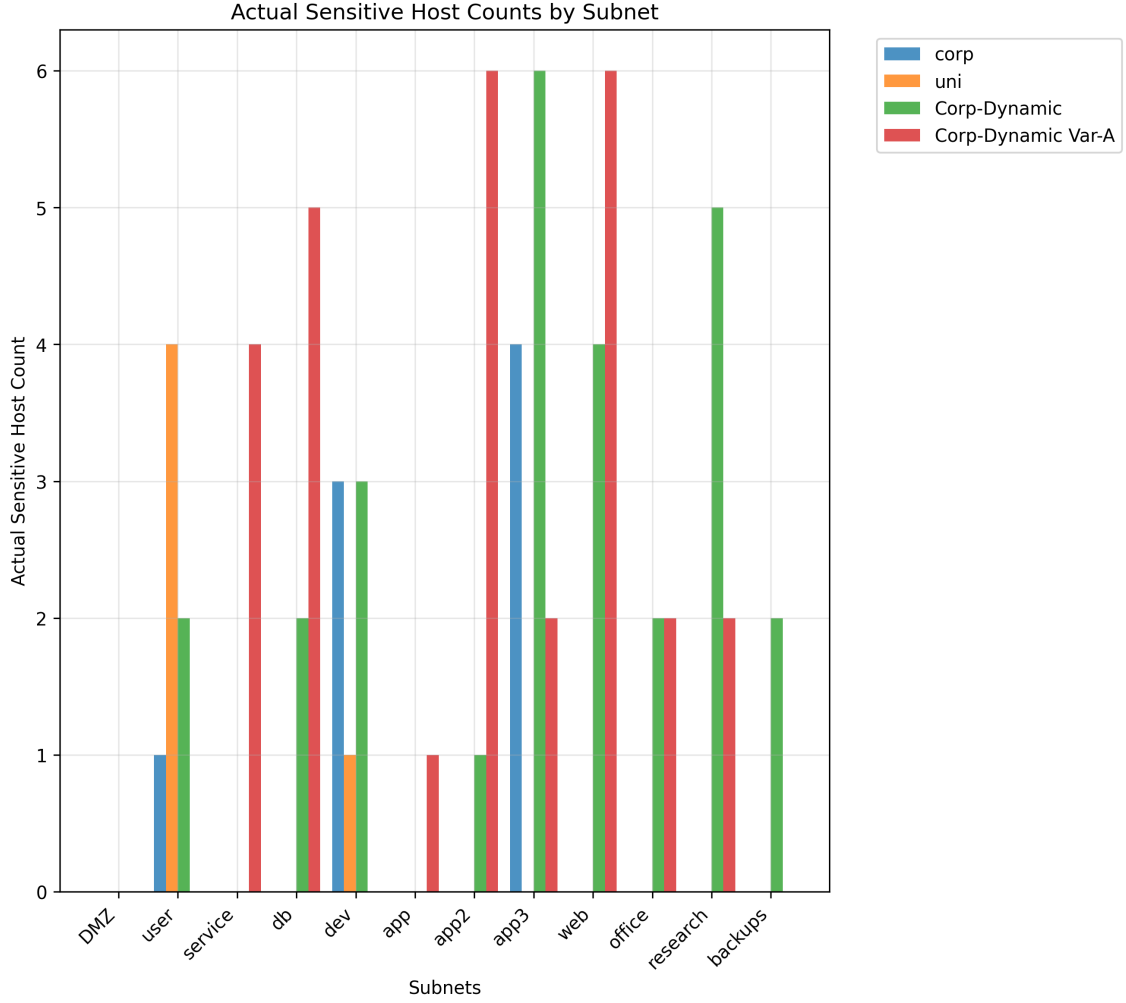


Figure 5.8: Sensitive Hosts Distribution Across Subnets

Costs, Noise, and Implications

- **corp**: Costs/Noise: unit scan costs; no explicit scan noise, churn, or timeouts. Implication: compact network with a minimal attack surface.
- **uni**: Costs/Noise: unit scan costs; no explicit scan noise, churn, or timeouts. Implication: slightly larger and more distributed than **corp.v2**, but still a baseline-sized attack surface.
- **corp_dynamic**: Costs/Noise: scan costs = 1; service churn active on selected services; network reliability timeouts for exploit/PE with retry semantics. Implication: larger attack surface, non-stationary services, and intermittent reliability create a robust training ground and meaningful generalization challenge.
- **corp_dynamic_varA**: Implication: reduced observability and tougher compromises elevate difficulty.

Cross-Scenario Comparison

- **Scale and connectivity**: $\text{corp} < \text{uni} < \text{dynamic} \approx \text{varA}$ in subnets and host counts. The larger topologies (dynamic, varA) present more traversal choices

and higher diameter/density variability; baselines are smaller and simpler to navigate.

- **Attack surface richness:**

- Baselines: 6 services, no processes, 5 exploits → narrow and stable.
- Dynamic/varA: 32 services, 13 processes, 8 exploits → rich, mixed OS stack; more lateral options and escalation routes.

- **Sensitive host distribution:**

- Baselines: focused in a few subnets (enterprise/academic conventional placements).
- Dynamic base: broader, high in **db**, **backups**, with elevated service, web, research—produces higher realized sensitive counts.
- VarA: Similar scale but different topology, reweighted sensitivities, increasing path-planning selectivity.

- **Difficulty knobs and realism:**

- Baselines: unit scans; no noise/churn/timeouts.
- Dynamic base & VarA: churn and timeouts introduce temporal variance and retries without misleading observations, explicit FP/FN scan noise degrade state estimation and inflate action budgets.

- **Derived indicators:**

- Services-per-host and exploits-per-host are markedly higher in dynamic/-varA, reflecting a denser per-node attack surface.
- Average exploit/PE probabilities and costs confirm the practical difficulty gap.

Table 5.5 represents the quantitative differences across key criteria and features in all four scenarios. There is an clear indication that the newly created scenarios can provide larger action space and state for the agent to learn in a wider variety. The increased hosts, processes and services numbers along with higher network diameter indicates a richer and more complex pathway for the agent to learn and explore.

Scenario	Subnets	Hosts	Services	Processes	Exploits	Sensitive Hosts	Svc/Host	Exp/Host	Net. Diameter	Net. Density
Corporate	8	21	6	0	5	7	0.29	0.24	5	0.25
University	9	27	6	0	5	3	0.22	0.19	4	0.27
Corp-Dynamic	12	83	32	13	8	39	0.39	0.10	6	0.19
Corp-Dynamic VarA	12	86	32	13	8	41	0.40	0.10	7	0.19

Table 5.5: Network scenario comparison: structural and security metrics.

Table 5.6 summarizes the variability of key network-level metrics across scenarios, highlighting their mean values, dispersion, and coefficient of variation. High coefficients of variation in Processes and Sensitive Hosts indicate substantial heterogeneity across scenarios, while lower variability in Exploits shows a stable exploit set for consistent action space and attack vectors.

Metric	Mean	Std Dev	Min	Max	CV (%)
Total Hosts	54.2	30.34	21	86	55.9%
Total Services	19.0	13.00	6	32	68.4%
Total Processes	6.5	6.50	0	13	100.0%
Total Exploits	6.5	1.50	5	8	23.1%
Total Sensitive Hosts	22.5	17.57	3	41	78.1%

Table 5.6: Summary statistics for scenario variability analysis.

5.5.2 RL architecture comparison

Original NasimEmu Architecture and Scenarios

The original NasimEmu has four RL architectures: MLP, GNN, Attention, and Invariant. Four models were trained on these architectures using `corp` as the training scenario and `uni` as the test scenario. Among them, GNN performed the best with the second highest capture rate, best reward, and most efficient episode length, while MLP performed the worst with the lowest capture and reward, taking the maximum steps for each episode without optimizing. The Attention model achieved the best average capture and reward in the training scenario but performed worse than GNN in the test scenario, showing less retention.

Figure 5.9 shows the values achieved by each original RL models in the static scenarios across average reward step, average reward per episode, average episode length and in average capture.

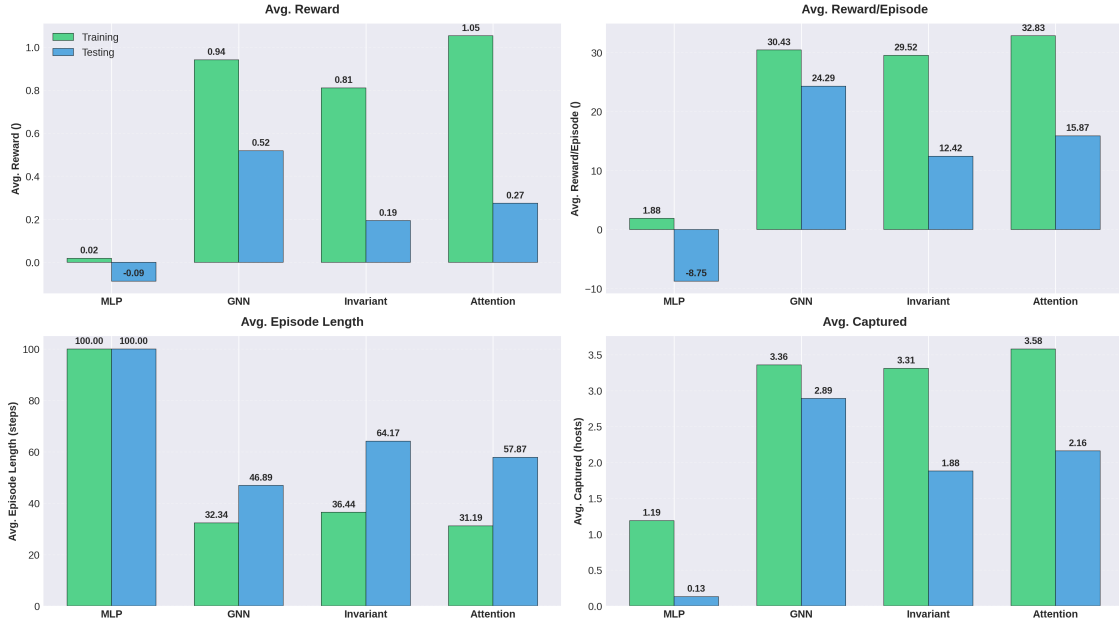


Figure 5.9: Model performance in original scenarios and architecture

Table 5.7 shows the generalization achieved by MLP, GNN, Attention, Invariant on static training and test scenario. There is a clear indication that the MLP

architecture overall less suitable to learn on it's on as it lacks to optimize for the better reward while loosing capture capability. On the other hand, Attention model while performed well in getting highest reward and capture in training scenarios it lacks generalization capability and overall GNN architecture is best suited for generalizing across different scenarios.

Architecture	Reward	Reward/Ep	Episode Length	Capture
MLP	-467%	-467%	100%	11%
GNN	55%	80%	145%	86%
Attention	24%	42%	176%	57%
Invariant	26%	48%	185%	60%

Table 5.7: Generalization comparison of RL architectures in the original NasimEmu scenario.

Staic Scenario and Feudal HRL Varaints

When both Feudal models were evaluated in static NASimEmu environments, baseline Feudal GTM performed a little better overall reward and a little shorter episode lengths than Dynamic Feudal GTM (DynFeudalGTM). This result matches theoretical expectations of their design architectures.

The baseline Feudal GTM was designed for static and fully observable environments with a single Graph Attention Layer (GAT) and light GRUCell memory. Since there are no stochastic or service churn in static environments, this simpler architecture is able to learn more quickly and take advantage of deterministic state-action patterns effectively. The fixed Manager-Worker coordination and fewer parameters prevent unwanted overfitting to temporal noise that is nonexistent in static episodes.

The Dynamic Feudal GTM, on the other hand, adds more temporal abstraction with two GAT layers and an LSTM-based Graph Temporal Memory. These add more flexibility in non-stationary networks, they also render the optimization process slower and more complicated under fixed conditions. As static environments do not need either long-term temporal reasoning or probabilistic control, the LSTM and multi-hop GAT layers would capture redundant relations, slowing slightly the convergence speed and final reward. Moreover, the increased model capacity brings greater risk of minimal overfitting during training caused by the absence of environmental randomness

Figure 5.10 shows the performance of GNN, Invariant, Attention FeudalGTM, Dyn-Feudal in the static scenarios across average reward step, average reward per episode, average episode length and in average capture.

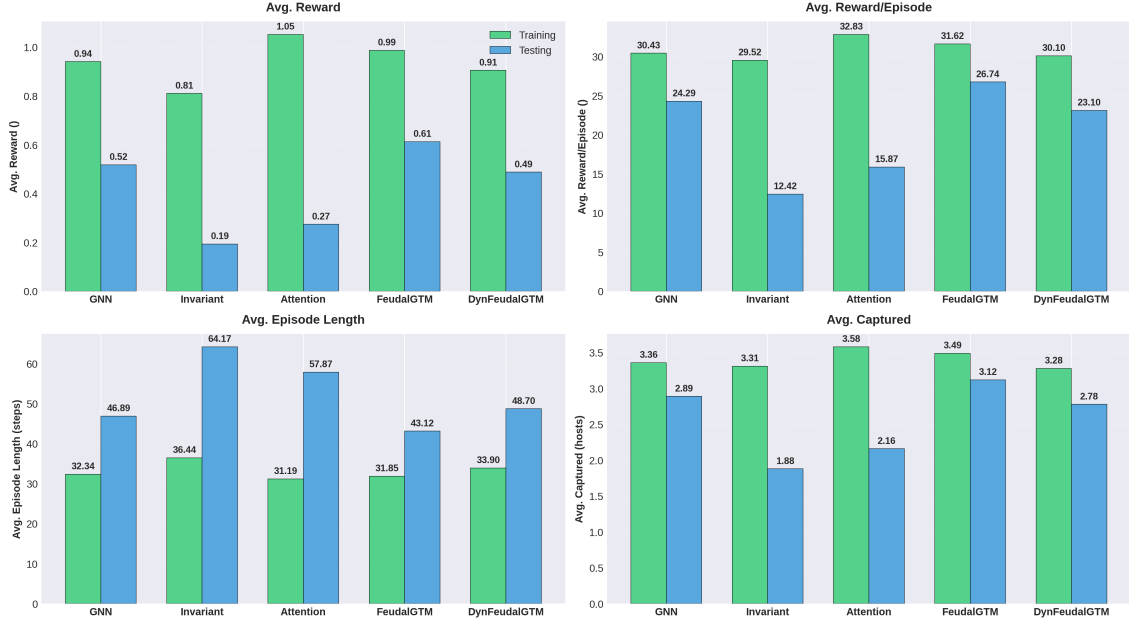


Figure 5.10: Model performance comparison of Feudal Variants in original static scenarios and architecture

Table 5.8 shows the of generalization achieved by MLP, GNN, Attention, Invariant, FeudalGTM, DynFeudalGTM on static training and test scenario. The simple feudal architecture was able to generalize capture learning better than the previous best model GNN in this criteria. In contrast, while less suitable for static scenarios Dynamic Feudal model was also on par with the GNN while outperforming Attention, Invariant in generalization.

Architecture	Reward	Reward/Ep	Episode Length	Capture
GNN	55%	80%	145%	86%
Attention	24%	42%	176%	57%
Invariant	26%	48%	185%	60%
FeudalGTM	54%	78%	135%	89%
DynFeudalGTM	54%	76%	143%	84%

Table 5.8: Generalization comparison of RL architectures in the original Static scenario.

New Scenarios and Dynamic Feudal HRL

In the next stage, `corp-dynamic` was used to train the models and `corp-dynamic-varA` was used to test retention and generalization in unseen scenarios. In these newly created larger, more difficult, and dynamic environments, GNN achieved the highest average capture, while Attention was the most efficient in terms of reward. The newly introduced Feudal HRL performed on par with GNN, achieving slightly lower capture and reward. In terms of episode length, GNN and Feudal HRL required

the maximum steps, while Invariant and Attention were more efficient but traded off between reward and capture.

Figure 5.11 shows the performance of GNN, Invariant, Attention, FeudalHRL in the dynamic scenarios across average reward step, average reward per episode, average episode length and in average capture.

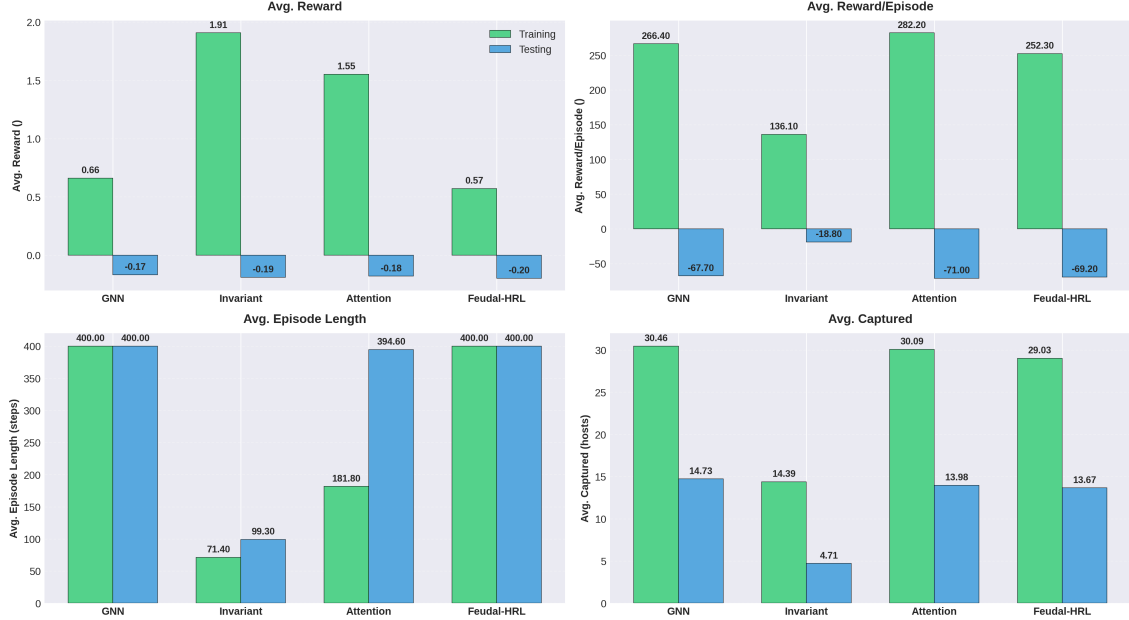


Figure 5.11: Original architecture and Feudal HRL performance in new scenarios

Table 5.9 shows the of generalization achieved by GNN, Invariant, Attention, FeudalHRL on dynamic training and test scenario. The largest difference appeared in test scenarios, where the retention of all models dropped significantly. This indicates that training on a single scenario does not enable a model to generalize effectively to a different unseen scenario.

Architecture	Reward	Reward/Ep	Episode Length	Capture
GNN	-25.75%	-25.24	100%	48.3%
Invariant	-9.95	-13.81	139.1%	32.7%
Attention	-11.61	-25.16	217%	46.4%
Feudal HRL	-50.87	-27.43	100%	47%

Table 5.9: Generalization of RL architectures including Feudal HRL in new dynamic scenarios.

Auto Scenario Generation

Finally, instead of training on a single fixed scenario, models were trained using template-based auto scenario generation. In this setup, a new scenario was generated in each episode, allowing the models to learn across a wide range of environments. Although average reward dropped significantly (as models could not optimize

against a fixed environment), both average capture and episode length improved, and generalization increased substantially.

To maintain consistency, **corp-dynamic** was still considered as the training scenario and **corp-dynamic-varA** as the test scenario. Feudal HRL achieved relatively better reward in this setting, consistent with its theoretical motivation.

Figure 5.12 shows the performance of GNN, Invariant, Attention, FeudalHRL in the dynamic scenarios trained with template based auto scenario generation across average reward step, average reward per episode, average episode length and in average capture.

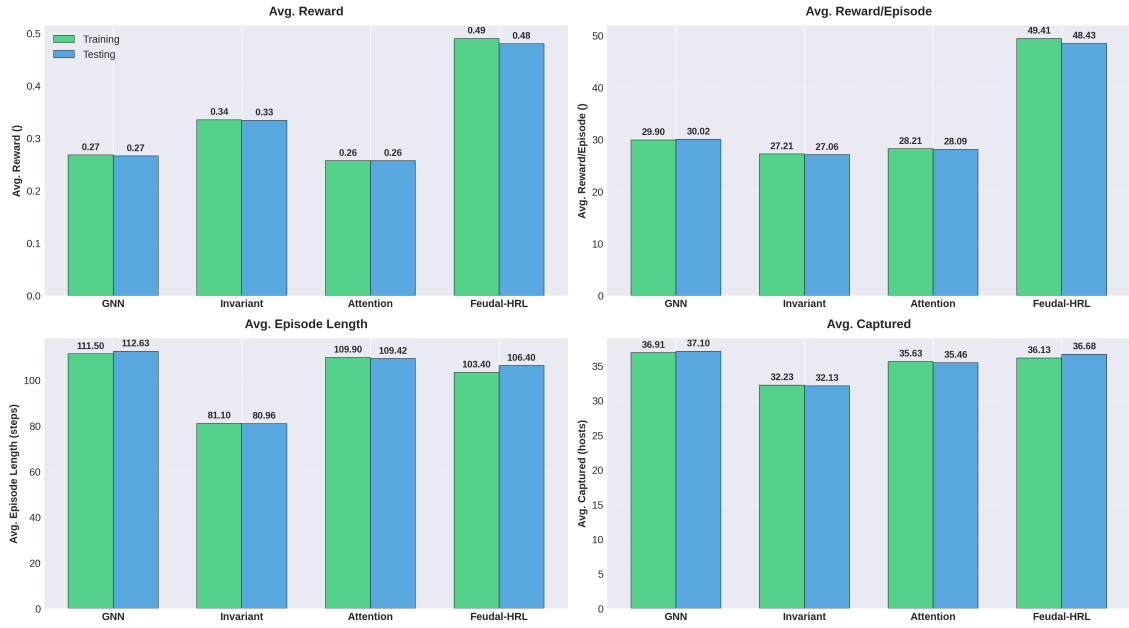


Figure 5.12: Original architecture and Feudal HRL performance in new Scenarios under auto scenario generation.”

Table 5.10 shows the of generalization achieved by GNN, Invariant, Attention, FeudalHRL trained with template based auto scenarios per episode and tested on two different dynamic scenarios. The key advantage was improved retention and generalization: all architectures demonstrated stable results across significantly different scenarios.

Architecture	Reward	Reward/Ep	Episode Length	Capture
GNN	100%	100%	99%	101%
Invariant	97%	99%	100%	100%
Attention	100%	100%	100%	100%
Feudal HRL	98%	98%	97%	102%

Table 5.10: Generalization of RL architectures under auto scenario generation training.

5.6 Discussions

This section includes a detailed description of the test outcome, observing impacts of the reinforcement learning (RL) architecture choice, architectural differences, and environmental complexities. Furthermore, the discussion delves on the broader observations pertinent to scalability, generalization and limitation in the proposed framework.

5.6.1 Interpretation of Results

The analysis demonstrates that the attacking method planned for reinforcement learning is skilled in obtaining viable exploitation sequences in fluctuating and diverse network environments. The analytical reward model (Eq. 5.2) strongly corresponds to the empirical outcomes, and therefore, it verifies the theoretical action-reward relationship and checking that the policies learned converge near-successful functioning within established budgetary limits.

The metrics of training—average reward, reward per episode, capture rate, and episode length, collectively uncover a systematic advancement in learning. At the outset, the agent concentrates on identifying and seizing hosts behavior, characterized by an increase in average captures in the beginning and followed by an extension in- The observed increase in rewards signifies a temporal decoupling, suggesting that the agent initially develops the exploration policy essential for consistent compromise prior to tuning for maximising efficiency. Convergence is characterised by stabilization of episode lengths and fixed per-step rewards, ensuring that the agent converges towards both operationally competent and policy-coherent governance structures

5.6.2 Architectural Insights

With different architectures, the performance indicates certain sacrifices among exploration breadth, representational richness, and policy efficiency. The GNN-based model consistently achieved the highest capture rates and overall gains, supporting the fact that graph-oriented descriptions are specifically suitable for well structured, relation-rich domains, such as network penetration. Its ability to generalize across topologically diverse scenarios uphold the hypothesis that state abstraction strengthens embeddings of graphs and transferability.

The Attention-based model exhibited superior efficiency in reward per episode and shorter episode lengths, suggesting higher selective focus and exploitation efficacy. It, however, underperformed constructed GNN in novel settings, suggesting too much specialisation towards environment-specific attributes. The Invariant model achieved average performance but showed significant degradation under high variability, indicating robustness without strong generalization. The Feudal HRL architecture—proposed for control hierarchies—acted in the same way as GNN, with marginally decreased reward but enhanced flexibility in the auto-scenario generation setting. Its hierarchical decomposition of objectives facilitated better temporal abstraction, permitting steadiness in non-stationary environments

5.6.3 Scenario and Generalization Behavior

Analysis of scenarios (Table 5.5) shows sensitivity of policy performance to environment complexity and stochasticity. Transitioning from the baseline (corp, uni) to the dynamic scenarios brought in higher topology size, service diversity, and probabilistic noise. These modifications significantly lowered intrinsic performance for all architectures when exposed to training in one environment. This points to an important limitation: traditional reinforcement learning training does not generalize appropriately to unknown networks. Distributions.

However, the use of automated scenario generation mitigated this limitation. By varying the net-parameters over the duration of multiple episodes, the agents had experienced multiple topological distributions and vulnerabilities, for more effective and universal policy formulation. Under this regime, all architectures achieved near-uniform generalization retention (Table 5.10), vindicating the validity of Curriculum-oriented, domain-randomized training methods for adversary environments.

5.6.4 Performance Implications

From the systems perspective, both stability and efficiency are as important as maximizing reward. The late-training decrease in average episode length corresponds to the agent’s ability to converge objectives with fewer actions, directly reflecting improved cost-efficiency. Furthermore, stable post-convergence behavior that is not rewarded shows resistance to overfitting and oscillation in policy—a typical problem for sparse-reward, high-dimensional RL problems.

The correspondence between theoretical reward constraints and empirical findings further corroborates the accuracy of the uncertainty of the cost and reward shaping functions. $R_{\max}$ and observed R_{real} differences are within forecasted limits with action randomness and exploration cost, confirming that the model derives near-optimal reward paths in realistic environments.

5.6.5 Limitations of the Results

Despite the strong empirical results, several shortcomings remain. First, the reinforcement learning agents are evaluated largely through simulation; application in real-world settings would involve complementing with real-time dynamics and adaptive adversarial feedback. Secondly, although auto-scenario generation enhances generalizability; however, it does not address all the combinatorial variability present in real-world businesses environments. Further research should involve adaptive domain randomization with guidance from environment those involve complexity measurements like network entropy and gradients of densities.

Also, hierarchies like Feudal HRL hold great promise but require more delicate sub-Goal setting and reward breakdown to make full use of their potential. Adding model-based or hybrid reinforcement learning could further enhance sample efficiency and interpretability. Finally, multi-agent or adversary specifications could provide an enriched representation by simulating Defender behavior facilitates the development of the framework into a comprehensive testbed for cyber-autonomy.

Chapter 6

Conclusion

In cybersecurity research, automating penetration testing is still a pressing and appealing objective. Although previous automated approaches usually failed to produce agents that generalise beyond the limited, static scenarios they were trained on, manual red-team engagements are costly, time-consuming, and do not scale to the growing attack surfaces of contemporary enterprise, cloud, and IoT infrastructures. By offering a dual simulation–emulation architecture and a collection of baseline agent designs, NASimEmu has created a crucial platform for addressing this issue. A practical gap was left by the original framework’s emphasis on fixed scenarios and primarily “flat” learning architectures, though, as agents that function well in a controlled NASimEmu scenario frequently falter when faced with the unpredictability and partial observability of operational networks. This thesis fills that gap by (1) adding template-driven, automated dynamic scenario generation to the training pipeline and (2) creating and assessing Feudal HRL, a hierarchical reinforcement learning agent that breaks down a complex red-team campaign into reusable low-level worker tactics and strategic manager goals. Therefore, the work assesses not only which model achieves the highest reward in a solitary lab setting, but also whether agents trained in this manner maintain their competence in noisy, non-stationary, richer services and exploit environments—conditions that are more representative of real-world scenarios. The main conclusions are summarised in the sections that follow, along with a critical analysis of the limitations and a roadmap for future research directions.

6.1 Summary of Findings

A central empirical observation of this research is that environmental diversity during training is as important as architectural sophistication. Compared to the baseline corporate and university scenarios, corp-dynamic and corp-dynamic-varA, the dynamic scenarios created and employed in this work significantly increased the size and diversity of training topologies. increase hosts, service counts and process richness, and introduce churn, scan noise and exploit timeouts that together create a denser and temporally varying attack surface. These structural alterations are not merely aesthetic; they penalize policies that were trained deterministically on a single, static scenario by increasing path-selection choices, producing temporal failures

that penalize brittle action sequences, and increasing the cost of blind exploration.

The comparative baseline results supported previous findings when models were trained on the original NASimEmu scenarios: graph-aware models outperformed a simple MLP baseline in terms of learning effective policies. Nonetheless, the experiments that stress models with change provide the most insightful data. Training exclusively on one dynamic scenario (corp-dynamic) and then testing on a different, unseen dynamic scenario (corp-dynamic-varA) exposed the limitations of single-scenario training: all architectures suffered retention losses in the unseen test environment. To put it briefly, robust generalization cannot be achieved by training on a single large scenario.

The way Feudal HRL behaved in these tests makes the relationship between training regimen and architecture clear. Feudal HRL outperformed the original architectures in the "new scenarios" experiment in a number of metrics and needed 72 longer but more comprehensive episodes to reach sensitive targets. However, the auto scenario generation experiments yield the most convincing outcome. All architectures significantly enhanced generalisation and retention by training agents with a template-based auto scenario generator that created a new parameterised environment in every episode. Under auto scenario generation, Feudal HRL achieved near-top generalisation statistics alongside other powerful models, suggesting that exposure to distributional variability during training significantly reduces the generalisation gap.

When combined, these findings support two complex conclusions. First, the separation of manager and worker facilitates reusable tactics, makes long-term planning manageable, and aids agents in reasoning about subgoals like subnet compromise, pivoting, and persistence. These are some of the structural benefits of hierarchical abstractions like Feudal HRL. Second, and just as importantly, HRL's potential can only be fulfilled when the training procedure itself offers a sufficiently varied range of difficulties. When training with automated dynamic scenarios, Feudal HRL exhibits its strongest and most dependable behavior in practice. This is because the statistical benefits of broad, randomized training and the architectural advantages of HRL combine to produce robust, generalizable policies.

The way Feudal HRL behaved in these tests makes the relationship between training regimen and architecture clear. Feudal HRL outperformed the original architectures in the "new scenarios" experiment in a number of metrics and needed 72 longer but more comprehensive episodes to reach sensitive targets. However, the auto scenario generation experiments yield the most convincing outcome. All architectures significantly enhanced generalisation and retention by training agents with a template-based auto scenario generator that created a new parameterised environment in every episode. Under auto scenario generation, Feudal HRL achieved near-top generalisation statistics alongside other powerful models, suggesting that exposure to distributional variability during training significantly reduces the generalisation gap.

6.2 Contribution to the field

This research advances the field of automated penetration testing by bridging the gap between controlled-scenario experimentation and the complex, dynamic realities of operational networks. Existing work in this area has been mainly focused on static

and manually defined environments, resulting in brittle agents that were unable to generalize outside of their training environment. By integrating automated dynamic scenario generation into the NASimEmu framework, this work introduced a practical mechanism for simulating dynamicity in topology, host configuration, and exploit timing to get closer to reflection of the unpredictability of real-world enterprise and IoT networks. This contribution significantly enhances the realism and scalability of training pipelines for reinforcement learning-based red-team agents.

The development and evaluation of the Feudal HRL (Feudal Hierarchical Reinforcement Learning) agent is another significant contribution, which breaks down the intricate multi-level penetration testing jobs into high-level strategic goals and low-level operational strategies. This hierarchical method shows a procedural advancement over conventional flat architectures which enables agents to decide actions over longer horizons and reuse learned behaviors across different situations. Thus the model exhibits more interpretable intermediate decision-making. The study sets a new standard for structure-aware learning in automated offensive cybersecurity by proving that Feudal HRL maintains robust performance and flexibility under a range of training scenarios.

Third, the thesis contributes empirically to the understanding of generalization in reinforcement learning for cybersecurity. The experiments show that environmental diversity during training is as critical as model sophistication for achieving resilient agent behavior. The combination of Feudal HRL with template-based scenario randomization significantly improves cognition and flexibility in unknown and unseen test environments. This insight initiates a clear scope for future RL-based security research, highlighting hierarchical abstraction and distributional exposure as dual pillars of robustness.

Lastly, the research makes a methodological contribution to the community by offering a reproducible and scalable framework for evaluating penetration testing agents. The incorporation of NASimEmu’s simulation-emulation functionality with dynamic scenario generation, modular reward design, and structured HRL architectures lays the groundwork for future experimentation. This approach has the potential to inform future research on intelligent defenders, multi-agent red-blue dynamics, and realistic tool validation. Collectively, these endeavors push automated penetration testing closer to becoming functionally feasible and set the framework for scalable and intelligent security assessment systems.

6.3 Recommendations for Future Work

The limitations above naturally yield a research agenda with both near-term and medium-term goals. First, and most immediate, is emulation integration and validation. The thesis already lays out a hybrid plan in which the simulator runs most episodes while a fraction (for example, 10%) are executed in a Vagrant/VM-based emulator to validate realism and to compute empirical exploit success rates. Implementing this plan would involve automating scenario translation (from generator YAML to emulator templates), establishing a scenario cache and validator, and running a scaled emulation sweep for critical exploit classes. Doing so would verify the simulation’s fidelity, surface timing and orchestration issues that do not appear in

abstract simulation, and provide an empirical bridge to real-world tools.

Second, enrich firewall realism by moving from subnet-level to host-level firewall modeling. Practically, this requires augmenting scenario templates with per-host rule sets (allow/deny lists, stateful timeouts, NAT behaviors) and by instrumenting the environment to expose reachability semantics to the agent’s observation. This will increase realism and force the agent to reason about host-by-host reachability and privilege escalation paths, producing more operationally useful campaigns.

Third, reduce reward sensitivity via adaptive reward shaping and meta-learning. For example, an outer controller could modulate reward weights based on learning progress, or a small population of agents with perturbed reward settings could be evolved and combined using ensemble or distillation techniques. This reduces brittle dependence on one manually tuned reward function.

Finally, adopt practical roadmaps for resource-aware experimentation: since transformer/GNN experiments are costly, prioritize the incremental path outlined above and couple it with parallelized training and early stopping so computational budget is used efficiently. Where cost is prohibitive, smaller yet focused experiments can be highly informative without requiring massive clusters.

The thesis has shown that the route to practical, automated penetration testing is twofold: we must both structure agents so they can reason heterarchically about multi-stage attacks and we must expose those agents to the diversity and temporal variability they will encounter in the wild. The Feudal HRL architecture demonstrated that hierarchical decomposition helps control long horizons and creates reusable tactics; automated dynamic scenario generation demonstrated that exposure to distributional variability during training is a critical enabler of generalization. At the same time, this work is realistic about the remaining gap: simulation-driven success is an essential milestone, but emulation and real-tool integration, finer-grained host-level realism, parallelized training pipelines, and adaptive reward and defender models are necessary steps on the path to production-quality systems. The thesis therefore contributes not only specific experimental evidence in favor of HRL plus dynamic training, but also a practical blueprint — a prioritized roadmap — for the next stage of work that will translate lab success into operational value. The empirical comparisons, scenario statistics, and experimental design provide a solid evidentiary basis for both the claims of progress and the recommendations described here.

In sum, the research reported here advances the state of automated penetration testing by moving the community away from brittle, single-scenario benchmarks and toward a methodology that combines hierarchical reasoning with distributionally rich training. That combination produces agents that are more robust, interpretable, and closer to the kinds of behavior security practitioners need, while also exposing the research questions that remain. With methodical extension along the lines proposed above — especially emulation validation, host-level realism, scalable distributed training, and defender adversaries — this line of work can make automated penetration testing an increasingly reliable and practical tool in real security operations.

Bibliography

- [1] N. Rane and A. Qureshi, “Comparative analysis of automated scanning and manual penetration testing for enhanced cybersecurity,” in *2024 12th International Symposium on Digital Forensics and Security (ISDFS)*, 2024, pp. 1–6. DOI: 10.1109/ISDFS60797.2024.10527240.
- [2] Q. Li, R. Wang, D. Li, F. Shi, M. Zhang, A. Chattopadhyay, Y. Shen, and Y. Li, “Dynpen: Automated penetration testing in dynamic network scenarios using deep reinforcement learning,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 8966–8981, 2024. DOI: 10.1109/TIFS.2024.346195.
- [3] H. P. T. Nguyen, K. Hasegawa, K. Fukushima, and R. Beuran, “Pengym: Realistic training environment for reinforcement learning pentesting agents,” *Computers & Security*, p. 104140, 2024. DOI: 10.1016/j.cose.2024.104140.
- [4] T. Cody, P. Beling, and L. Freeman, “Towards continuous cyber testing with reinforcement learning for whole campaign emulation,” in *2022 IEEE AUTOTESTCON*, 2022, pp. 1–5. DOI: 10.1109/AUTOTESTCON47462.2022.9984769.
- [5] S. G. Bianou and R. G. Batogna, “Pentest-ai, an llm-powered multi-agents framework for penetration testing automation leveraging mitre attack,” in *2024 IEEE International Conference on Cyber Security and Resilience (CSR)*, IEEE, 2024, pp. 763–770.
- [6] V. Saber, D. ElSayad, A. M. Bahaa-Eldin, and Z. Fayed, “Automated penetration testing, a systematic review,” in *2023 International Mobile, Intelligent, and Ubiquitous Computing Conference (MIUCC)*, IEEE, 2023, pp. 373–380.
- [7] C. Szepesvári, *Algorithms for reinforcement learning*. Springer nature, 2022.
- [8] A. C. Moreno, A. Hernandez-Suarez, G. Sanchez-Perez, L. K. Toscano-Medina, H. Perez-Meana, J. Portillo-Portillo, J. Olivares-Mercado, and L. J. García Villalba, “Analysis of autonomous penetration testing through reinforcement learning and recommender systems,” *Sensors*, vol. 25, no. 1, p. 211, 2025.
- [9] V.-H. Pham, H. Do Hoang, P. T. Trung, T.-N. To, P. T. Duy, *et al.*, “Raiju: Reinforcement learning-guided post-exploitation for automating security assessment of network systems,” *Computer Networks*, vol. 253, p. 110706, 2024.
- [10] A. Samad, S. Altaf, and M. J. Arshad, “Advancements in automated penetration testing for iot security by leveraging reinforcement learning,” *evaluation*, vol. 8, p. 9, 2024.

- [11] M. C. Ghanem, T. M. Chen, and E. G. Nepomuceno, “Hierarchical reinforcement learning for efficient and effective automated penetration testing of large networks,” *Journal of Intelligent Information Systems*, vol. 60, no. 2, pp. 281–303, 2023.
- [12] T. Cody, E. Meno, P. Beling, and L. Freeman, “Whole campaign emulation with reinforcement learning for cyber test,” *IEEE Instrumentation & Measurement Magazine*, vol. 26, no. 5, pp. 25–30, 2023.
- [13] M. Alhamed and M. H. Rahman, “A systematic literature review on penetration testing in networks: Future research directions,” *Applied Sciences*, vol. 13, no. 12, p. 6986, 2023.
- [14] E. Filiol, F. Mercaldo, and A. Santone, “A method for automatic penetration testing and mitigation: A red hat approach,” *Procedia Computer Science*, vol. 192, pp. 2039–2046, 2021.
- [15] D. Kongara and S. Krishnama, “A process of penetration testing using various tools,” *Mesopotamian Journal of Cybersecurity*, vol. 2023, pp. 93–103, 2023. DOI: 10.58496/MJCS2023014.
- [16] M. Ashawa and S. J. Kpelai, “Penetration testing analysis of emerging technologies and their impact on pen testing,” *International Journal on Engineering Technologies and Informatics*, vol. 4, no. 4, 2023. DOI: 10.51626/ijeti.2023.04.00064.
- [17] H. J. Badarneh, L. A. Attiany, M. Asassfeh, A. Al-Shaikh, S. Afaneh, M. M. A. Shquier, N. Al-Milli, A. Elrashidi, and G. Samara, “The power of network penetration testing,” in *2024 25th International Arab Conference on Information Technology (ACIT)*, IEEE, 2024, pp. 1–6.
- [18] J. Janisch, T. Pevný, and V. Lisý, “Nasimemu: Network attack simulator & emulator for training agents generalizing to novel scenarios,” in *European Symposium on Research in Computer Security*, Springer, 2023, pp. 589–608.